Introduction to Deep Learning

Caltech CS/EE/CMS 148a - Winter 2026

Lecture 17 - Distillation, Teacher-Student, DINO

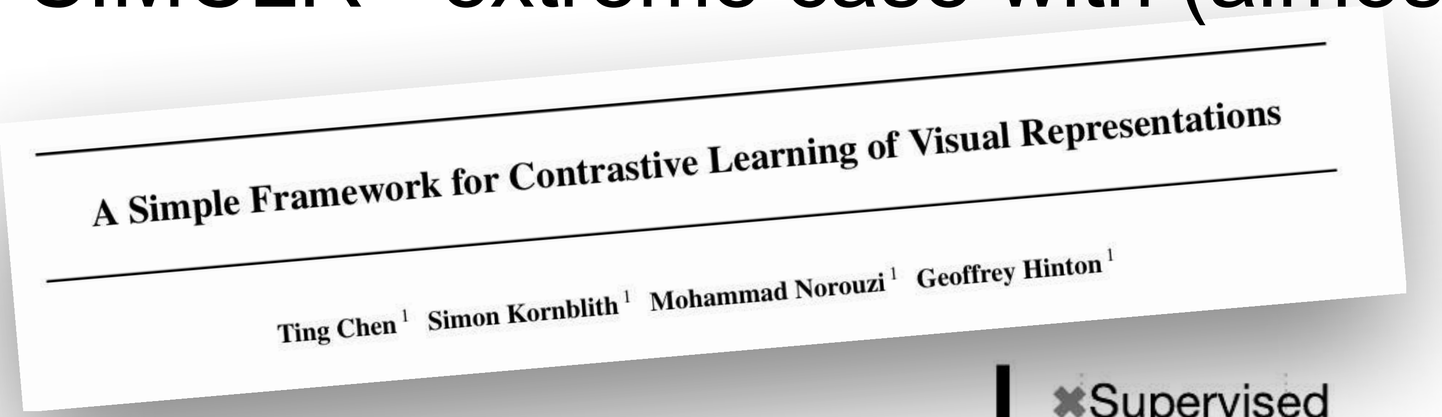
Self-supervised learning (Lecture 14)

Consistency regularization

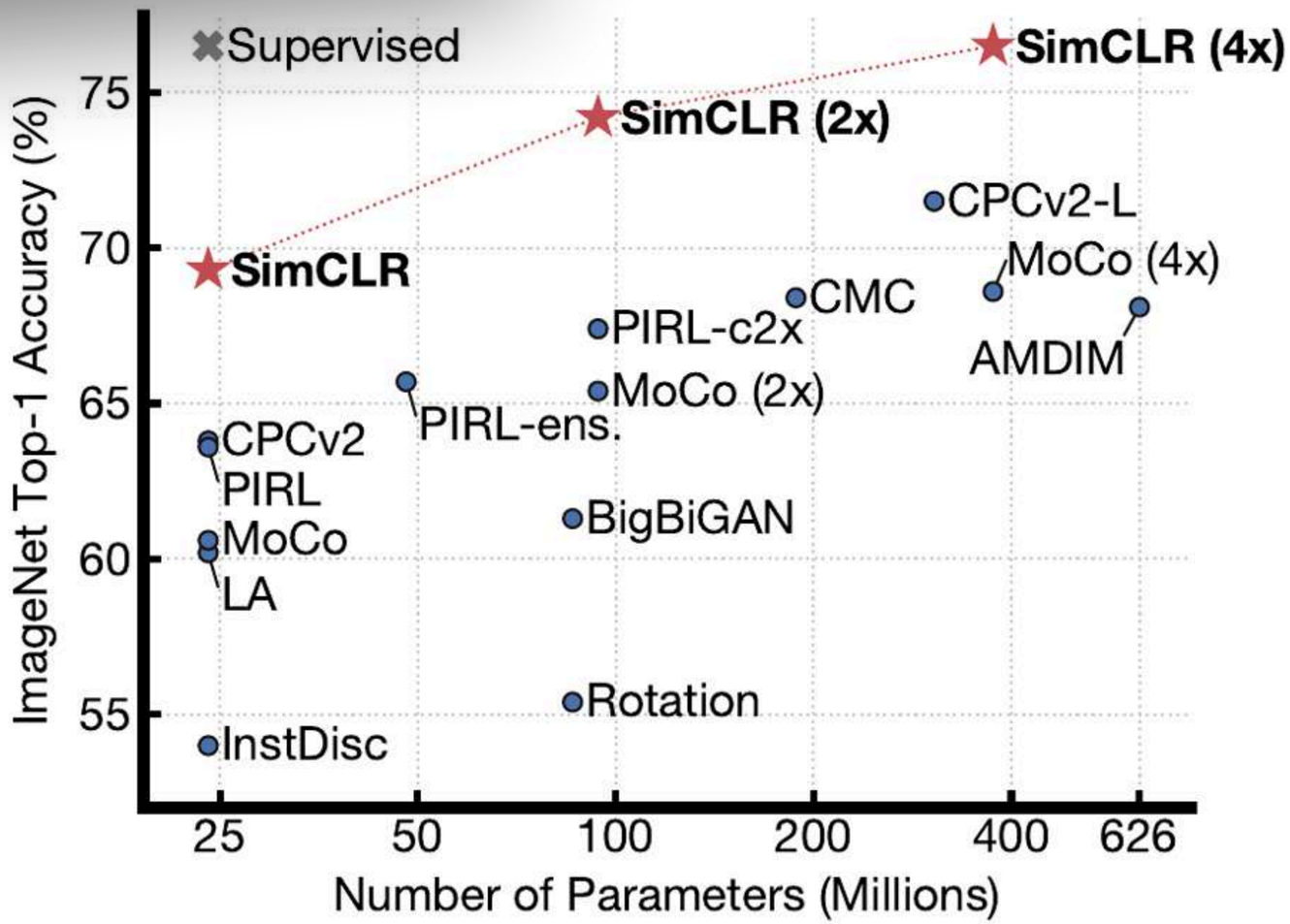
Enforce consistent prediction for similar inputs

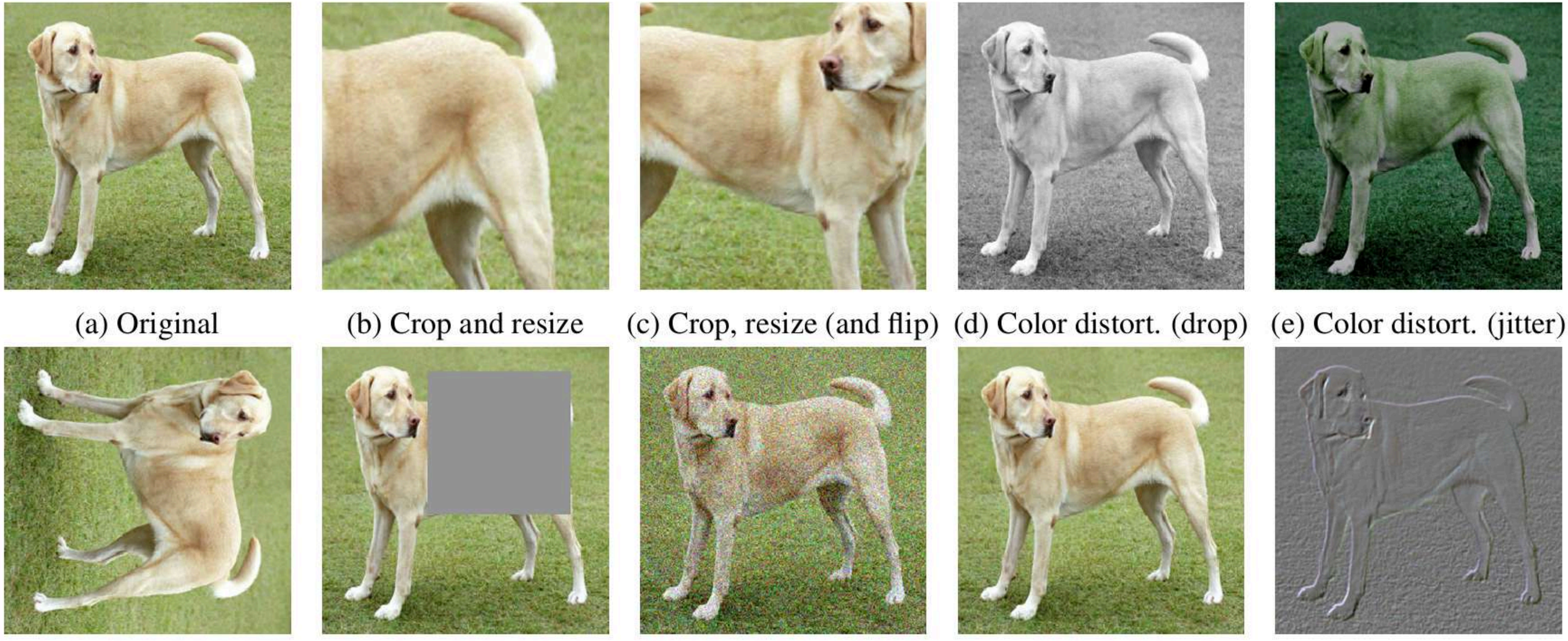
Add a consistency component to the loss function

SIMCLR - extreme case with (almost) no labels


A Simple Framework for Contrastive Learning of Visual Representations

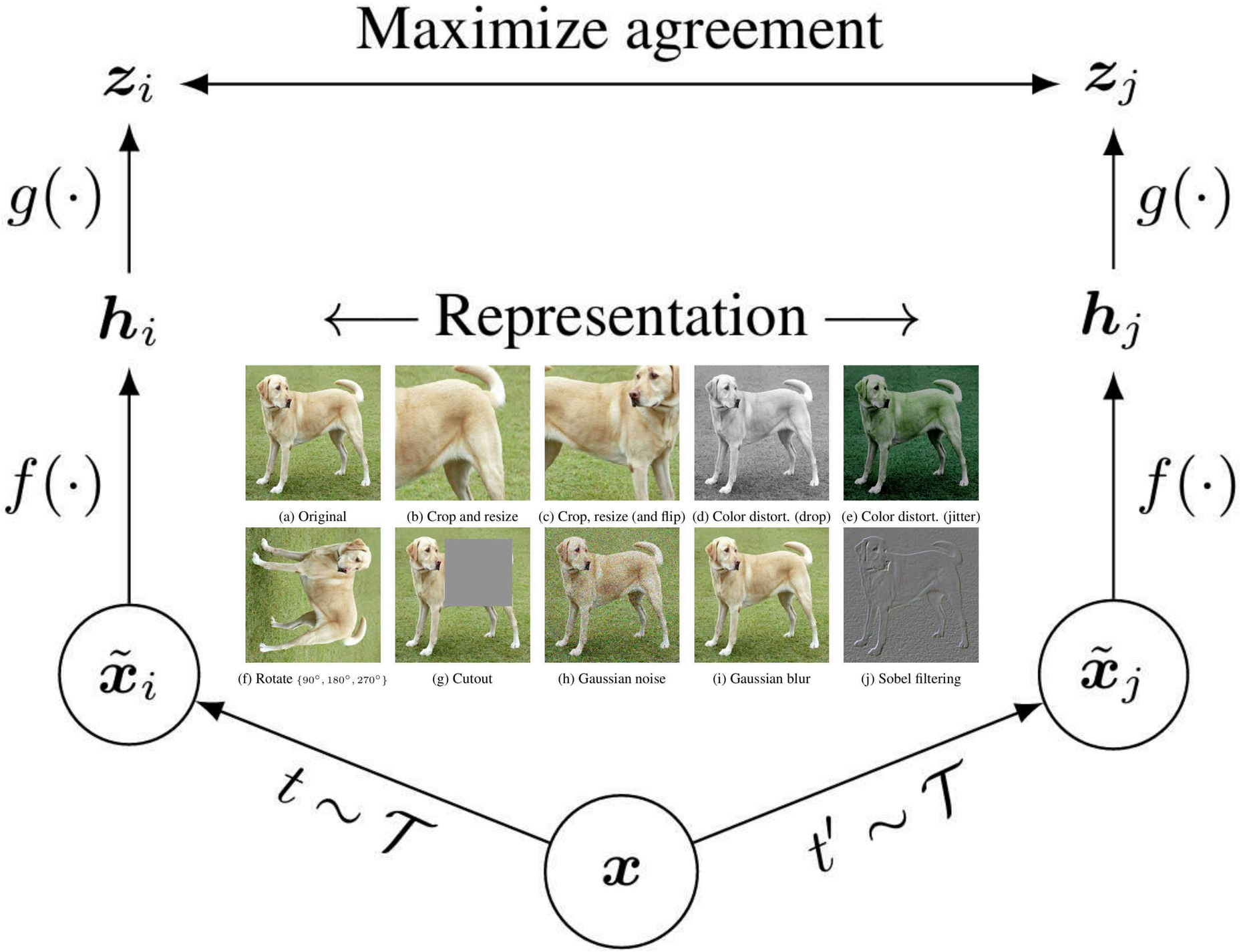
Ting Chen [1] Simon Kornblith [1] Mohammad Norouzi [1] Geoffrey Hinton [1]





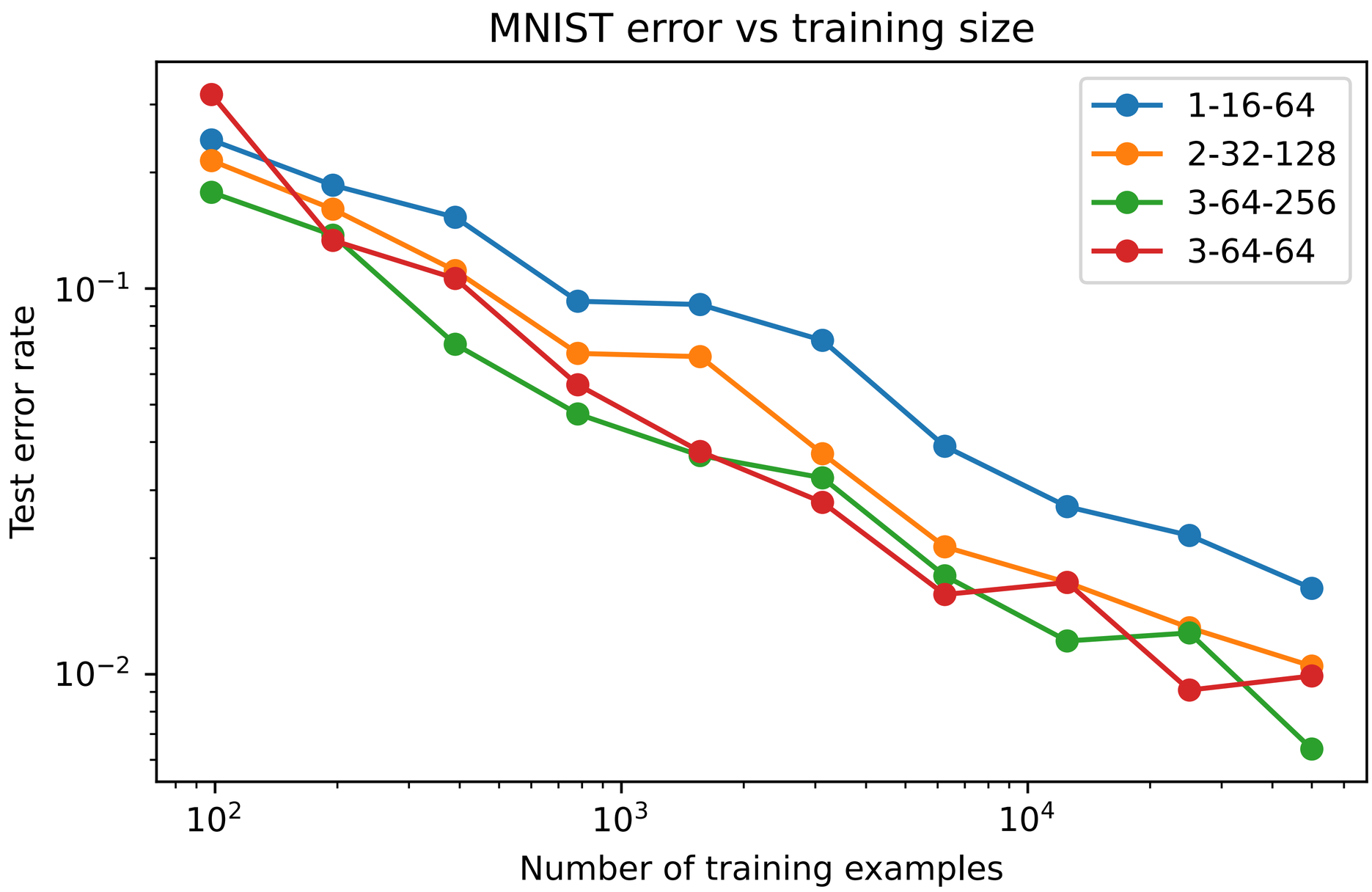
(a) Original (b) Crop and resize (c) Crop, resize (and flip) (d) Color distort. (drop) (e) Color distort. (jitter)

(f) Rotate $\{90°, 180°, 270°\}$ (g) Cutout (h) Gaussian noise (i) Gaussian blur (j) Sobel filtering

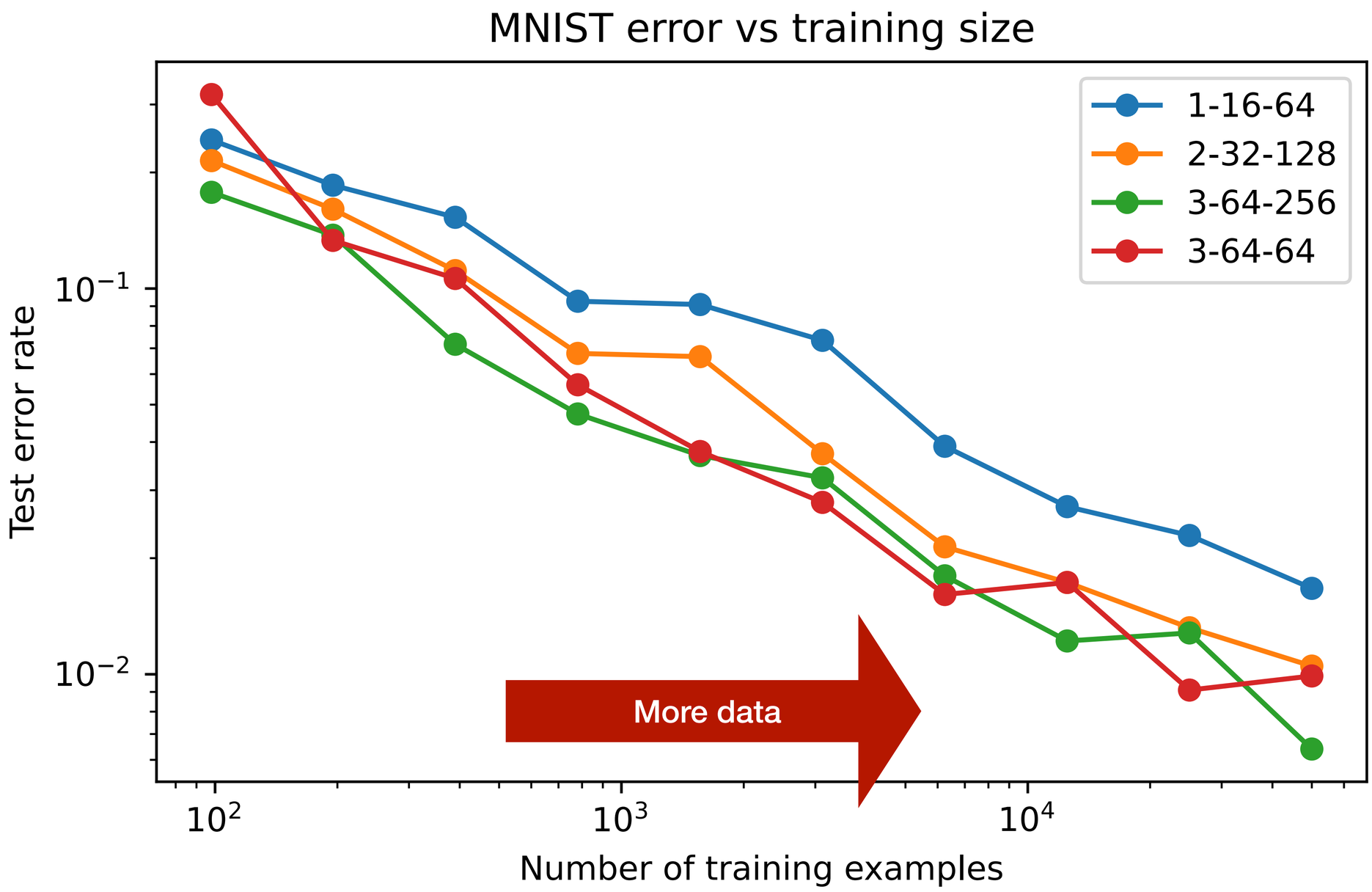

Maximize agreement
z_i
z_j
$g(\cdot)$
$g(\cdot)$
h_i
Representation
h_j
$f(\cdot)$
$f(\cdot)$
(a) Original
(b) Crop and resize
(c) Crop, resize (and flip)
(d) Color distort. (drop)
(e) Color distort. (jitter)
(f) Rotate {90°, 180°, 270°}
(g) Cutout
(h) Gaussian noise
(i) Gaussian blur
(j) Sobel filtering
$\tilde{x}_i$
$\tilde{x}_j$
$t \sim \mathcal{T}$
$t' \sim \mathcal{T}$
x

Distillation

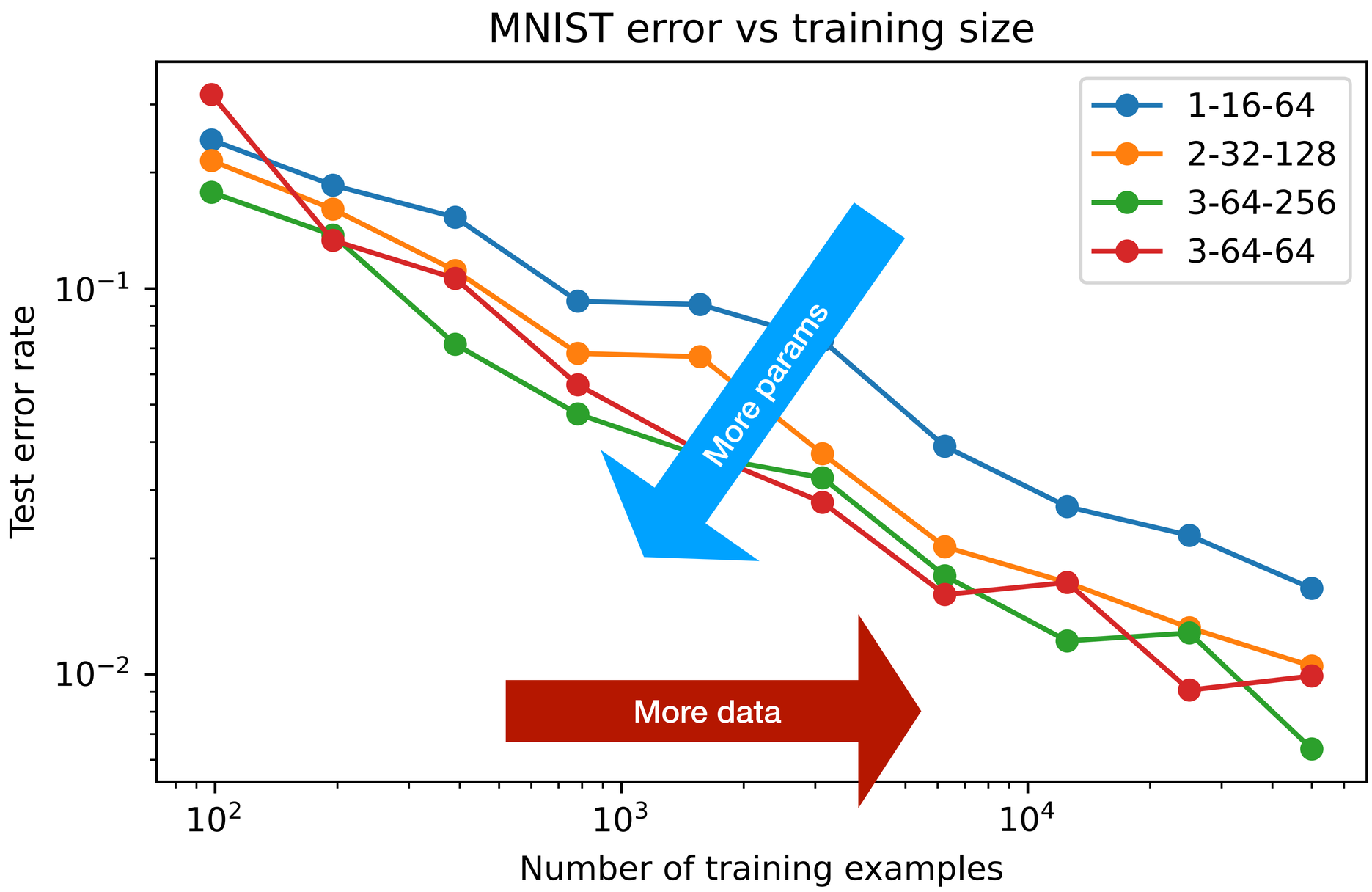
More data, more parameters => better accuracy

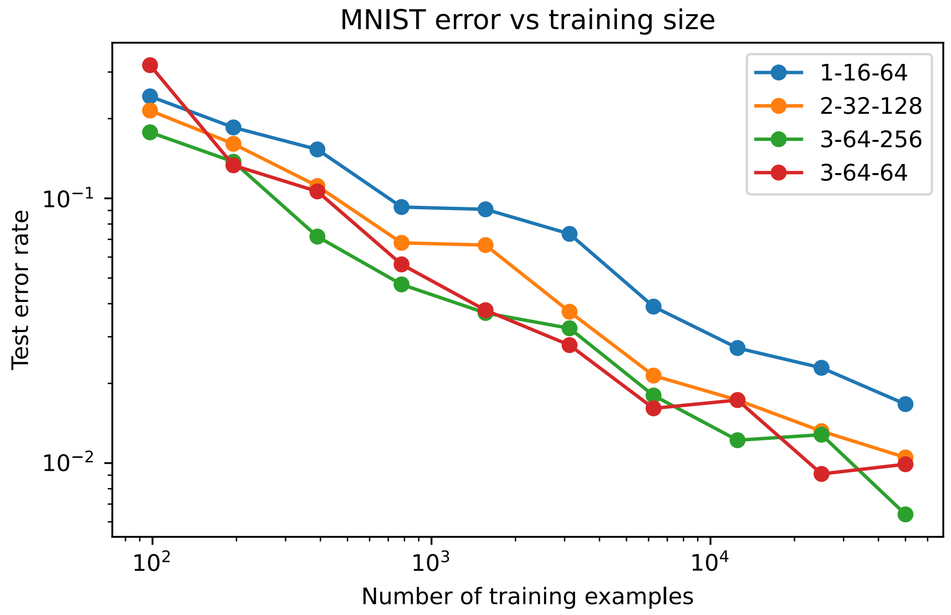


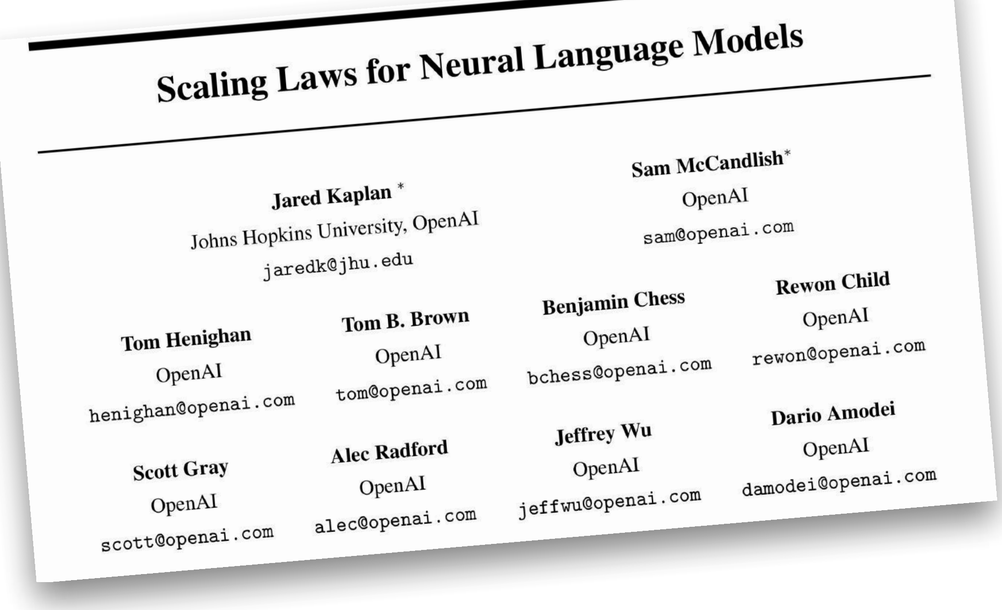
More data, more parameters => better accuracy



More data, more parameters => better accuracy




MNIST error vs training size
1-16-64
2-32-128
3-64-256
3-64-64
Test error rate
10^{-1}
10^{-2}
10^{2}
10^{3}
10^{4}
Number of training examples



Scaling Laws for Neural Language Models

Jared Kaplan *
Johns Hopkins University, OpenAI
jaredk@jhu.edu

Sam McCandlish*
OpenAI
sam@openai.com

Tom Henighan
OpenAI
henighan@openai.com

Tom B. Brown
OpenAI
tom@openai.com

Benjamin Chess
OpenAI
bchess@openai.com

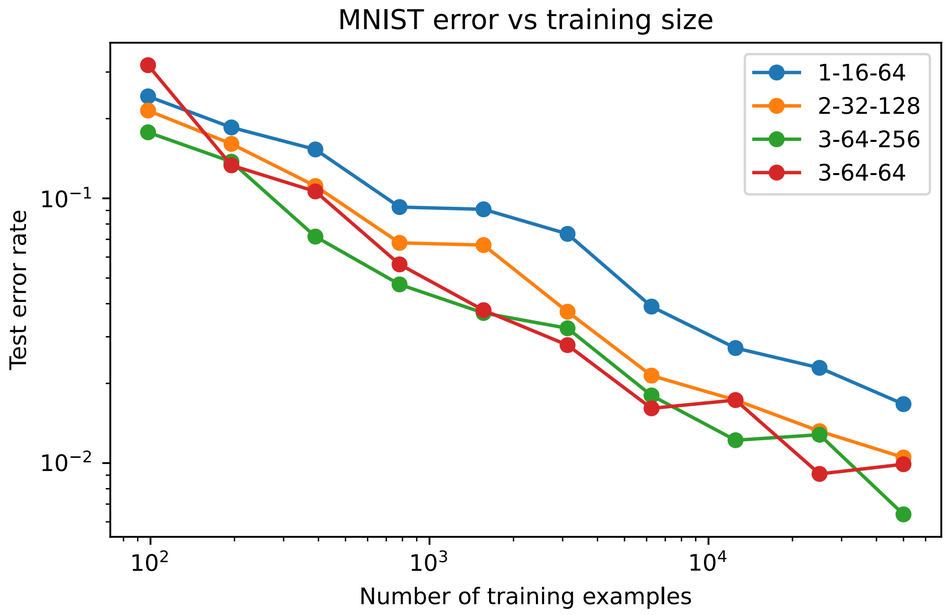
Rewon Child
OpenAI
rewon@openai.com

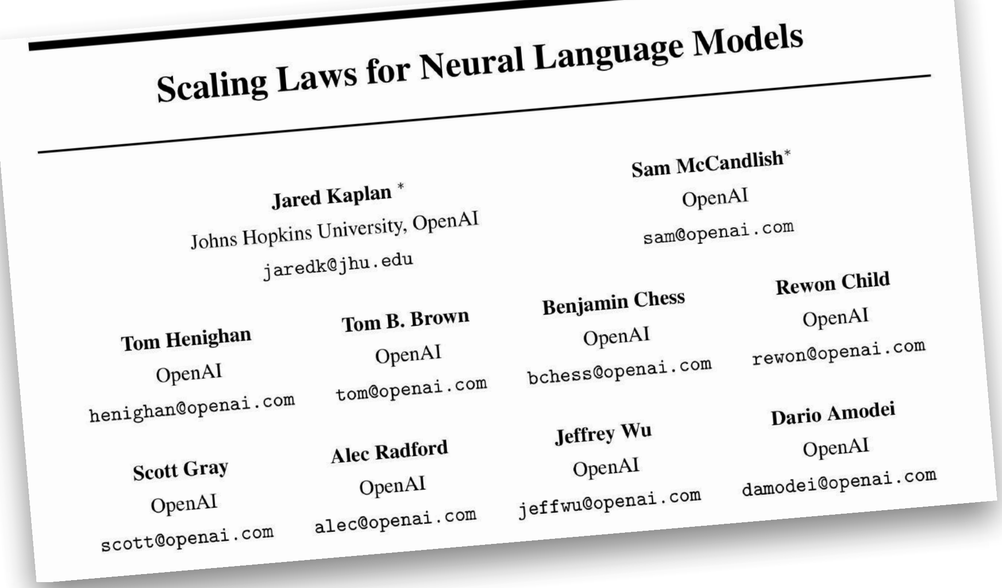
Scott Gray
OpenAI
scott@openai.com

Alec Radford
OpenAI
alec@openai.com

Jeffrey Wu
OpenAI
jeffwu@openai.com

Dario Amodei
OpenAI
damodei@openai.com





Scaling Laws for Neural Language Models

Jared Kaplan *
Johns Hopkins University, OpenAI
jaredk@jhu.edu

Sam McCandlish*
OpenAI
sam@openai.com

Tom Henighan
OpenAI
henighan@openai.com

Tom B. Brown
OpenAI
tom@openai.com

Benjamin Chess
OpenAI
bchess@openai.com

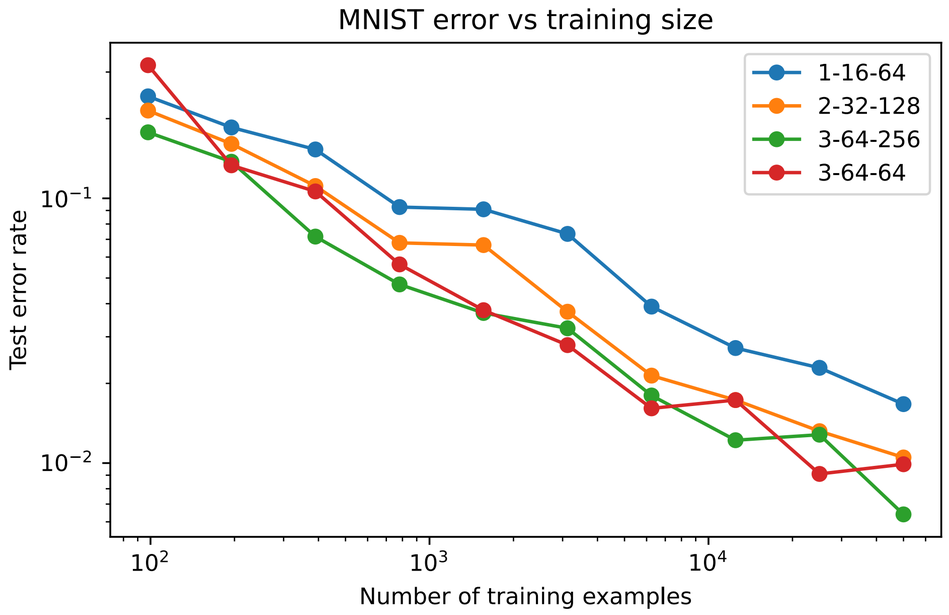
Rewon Child
OpenAI
rewon@openai.com

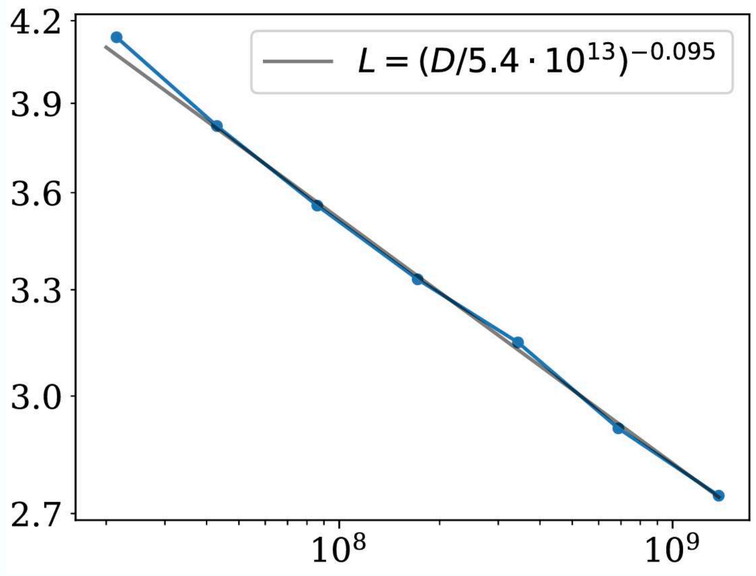
Scott Gray
OpenAI
scott@openai.com

Alec Radford
OpenAI
alec@openai.com

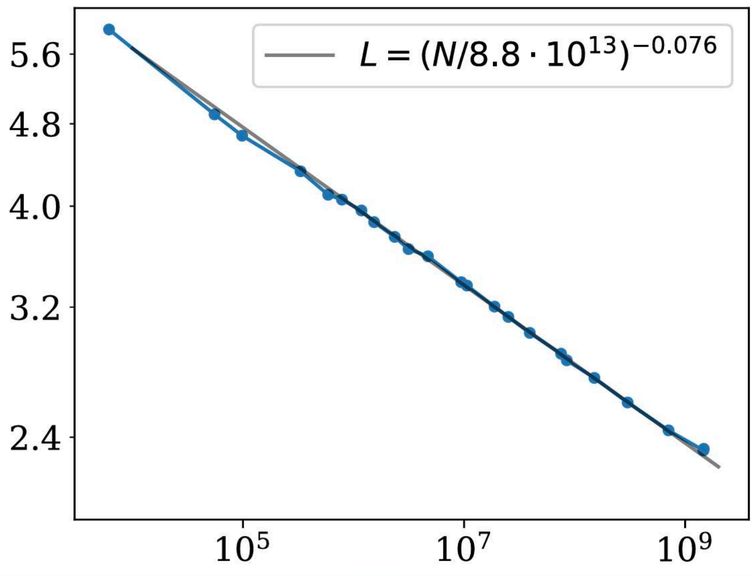
Jeffrey Wu
OpenAI
jeffwu@openai.com

Dario Amodei
OpenAI
damodei@openai.com



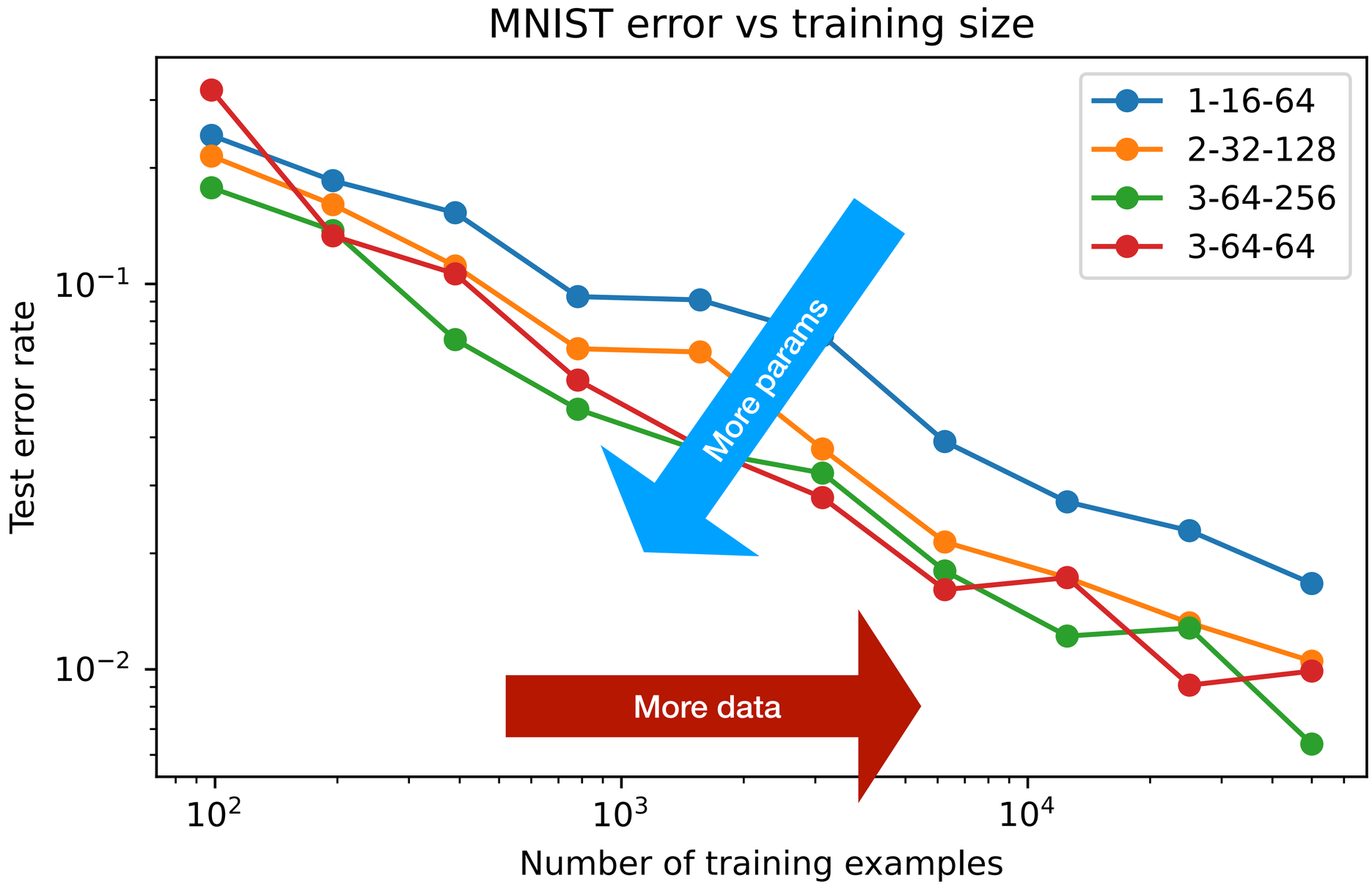


Dataset Size
tokens

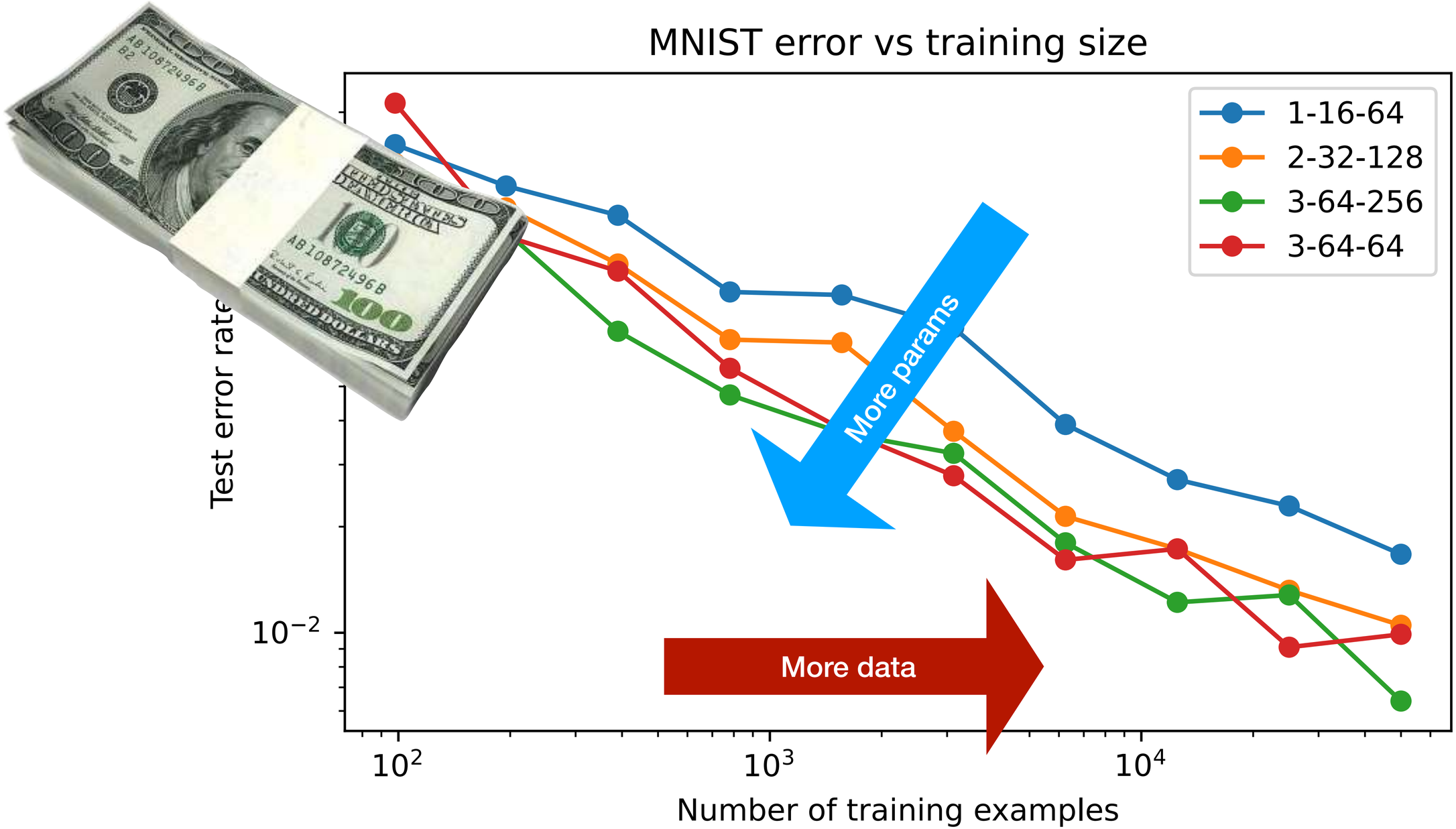


Parameters
non-embedding

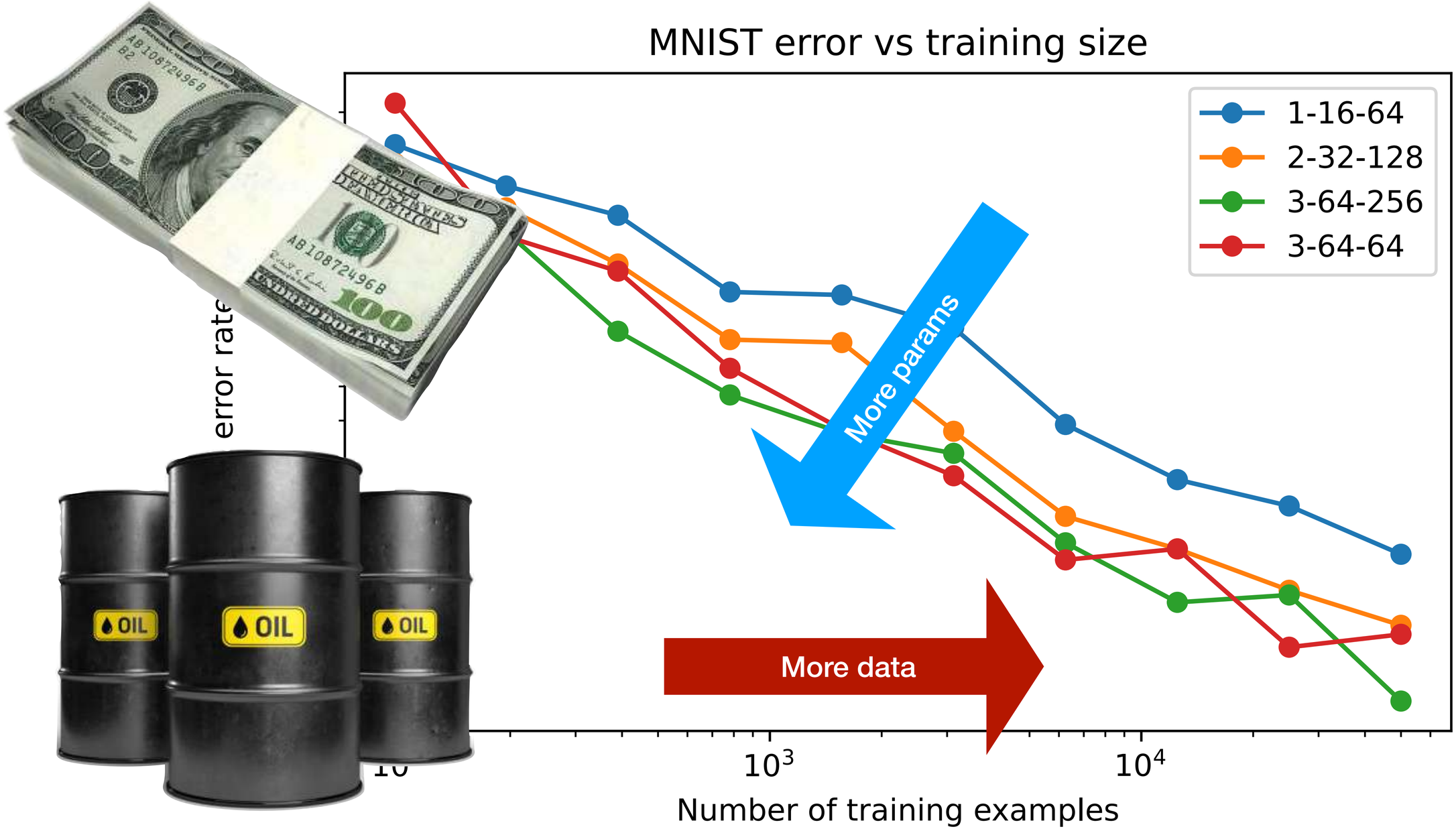
More data, more parameters => !!



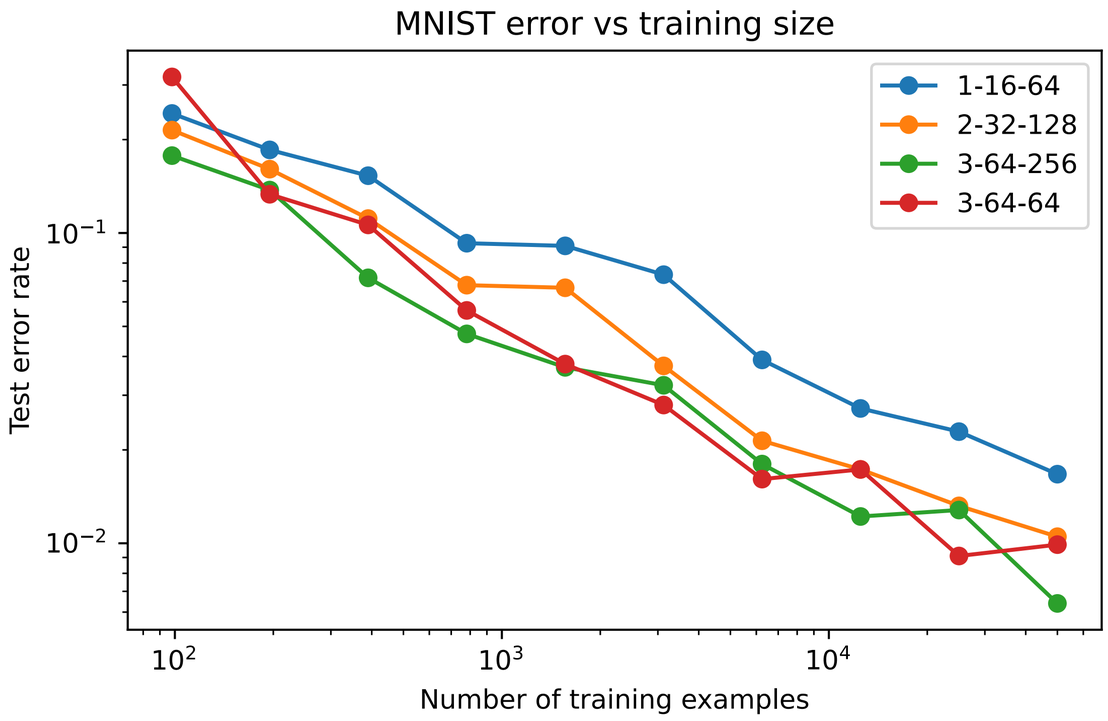
More data, more parameters => !!



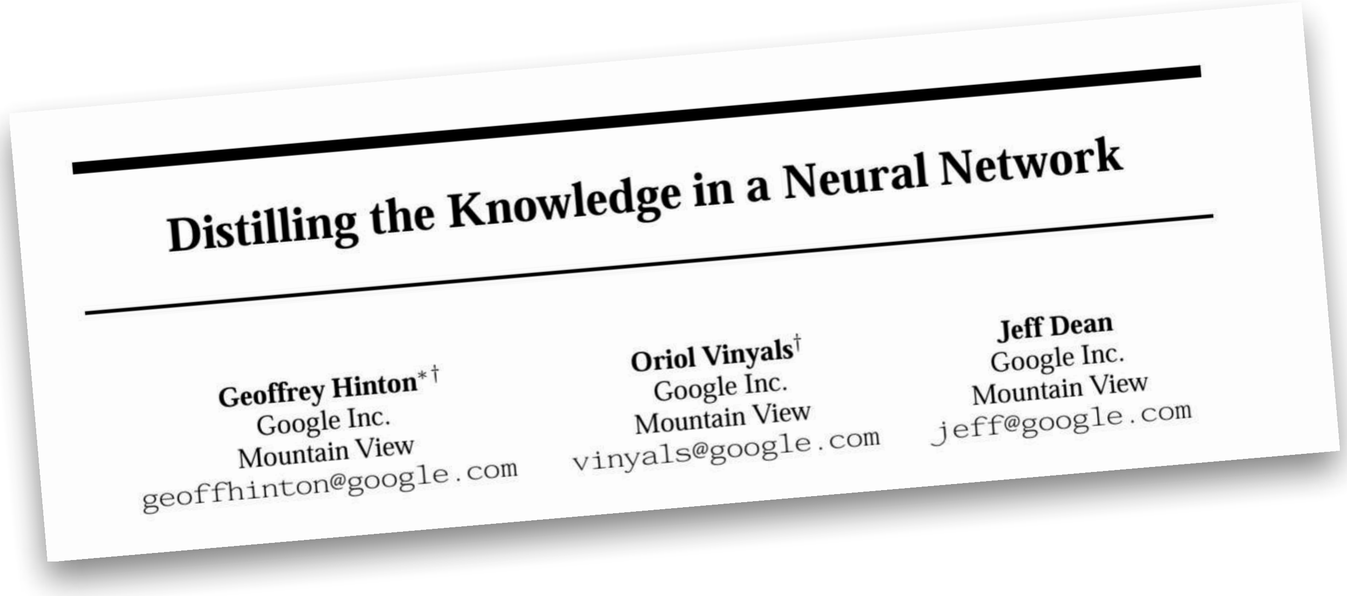
More data, more parameters => !!



Knowledge distillation



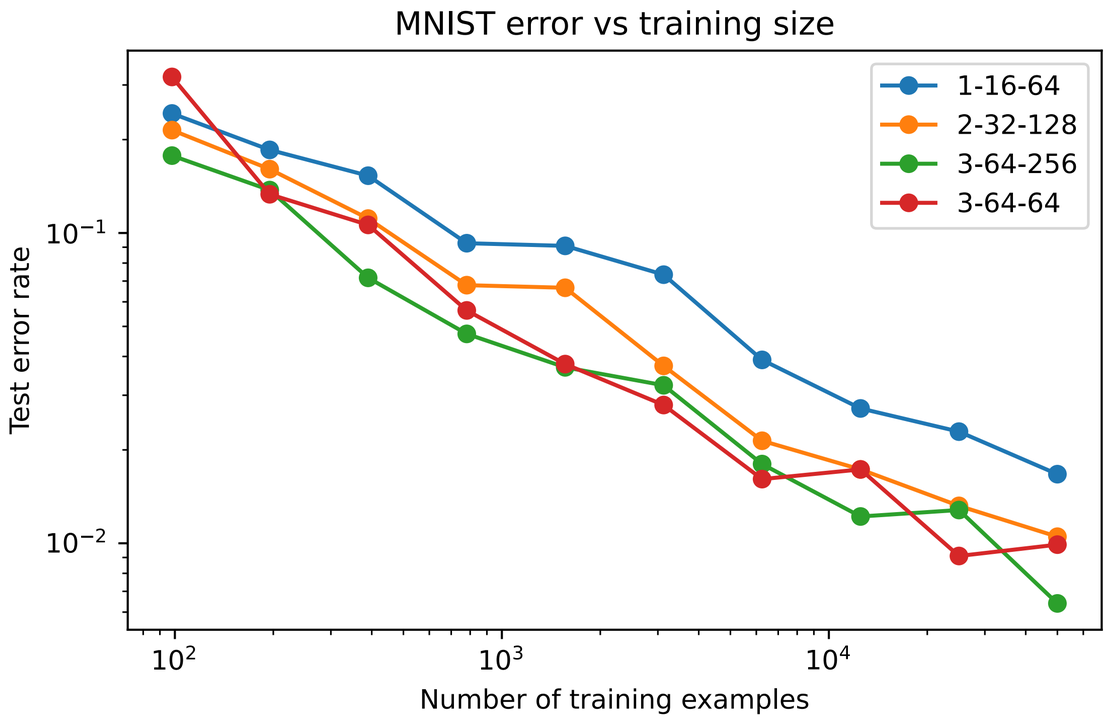
Knowledge distillation


Distilling the Knowledge in a Neural Network

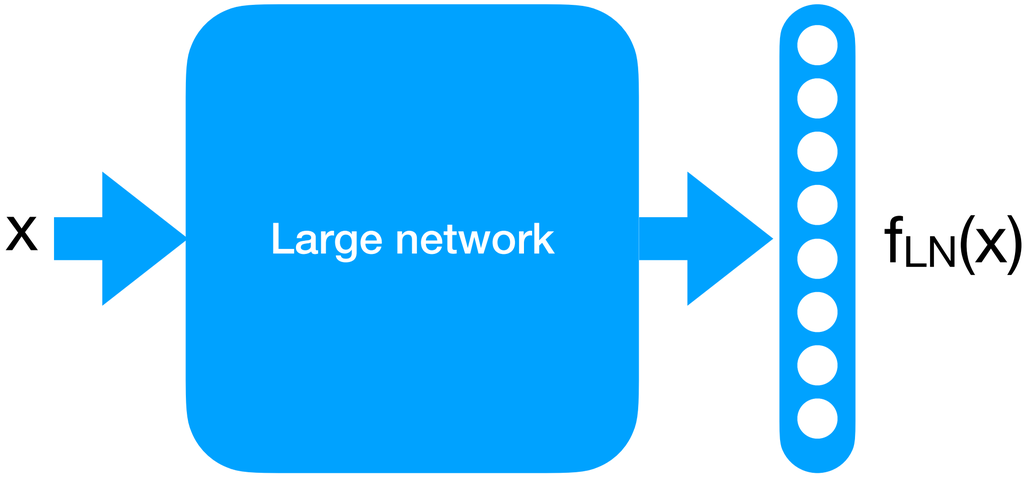
Geoffrey Hinton*†
Google Inc.
Mountain View
geoffhinton@google.com

Oriol Vinyals†
Google Inc.
Mountain View
vinyals@google.com

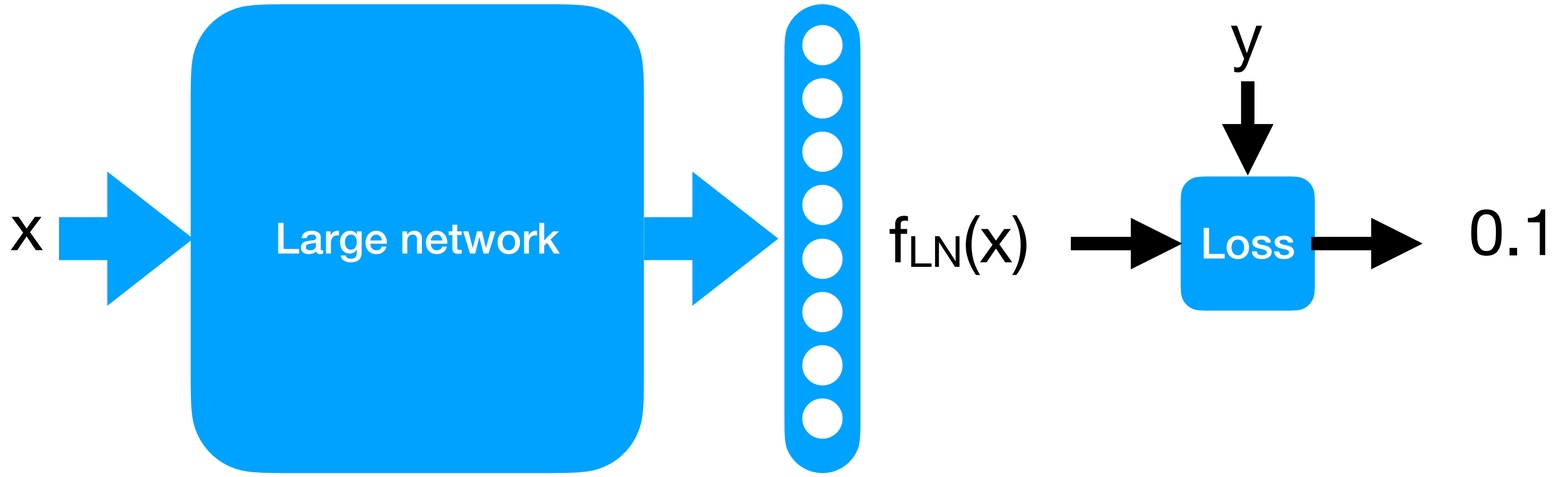
Jeff Dean
Google Inc.
Mountain View
jeff@google.com



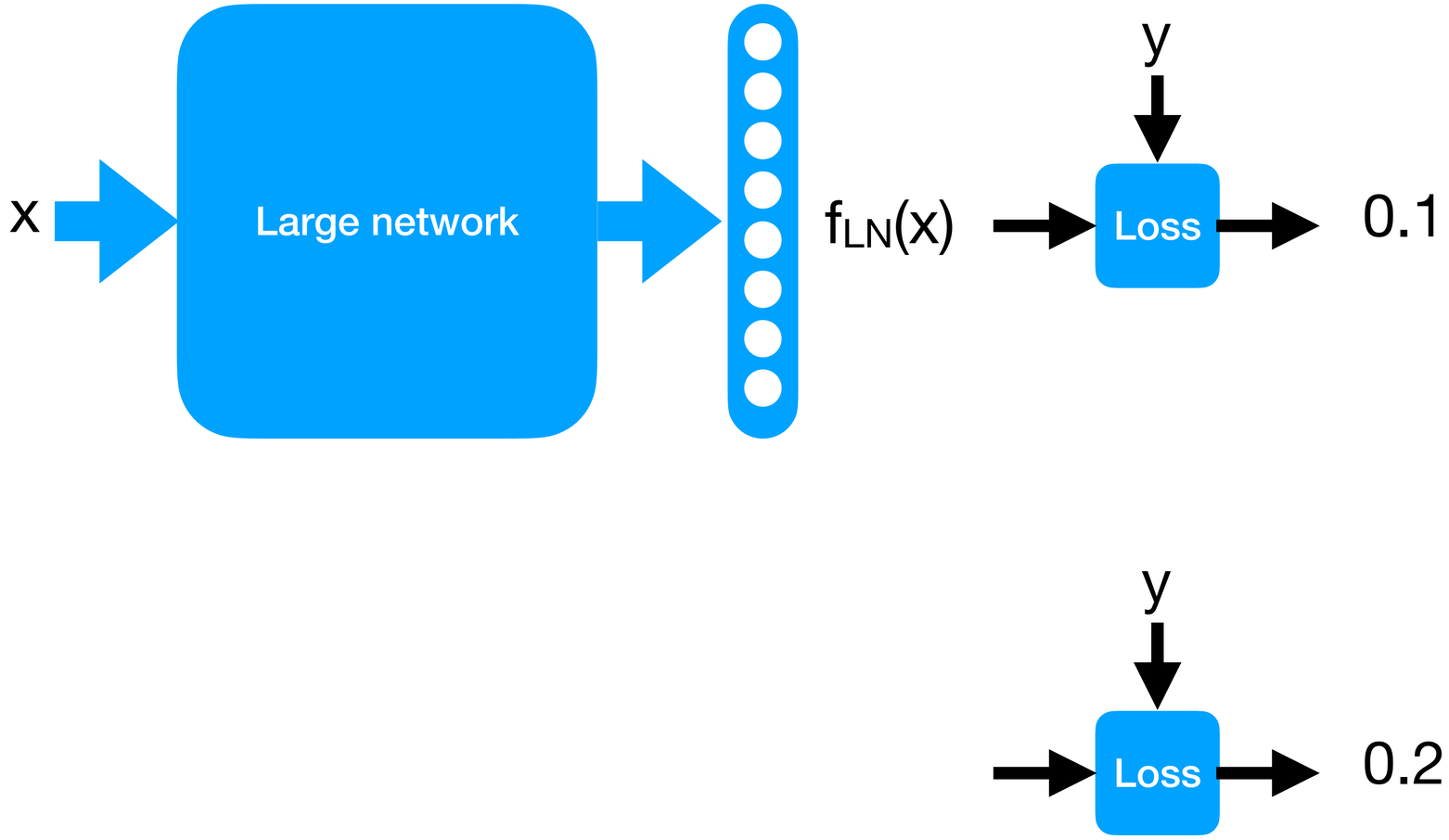
Knowledge distillation



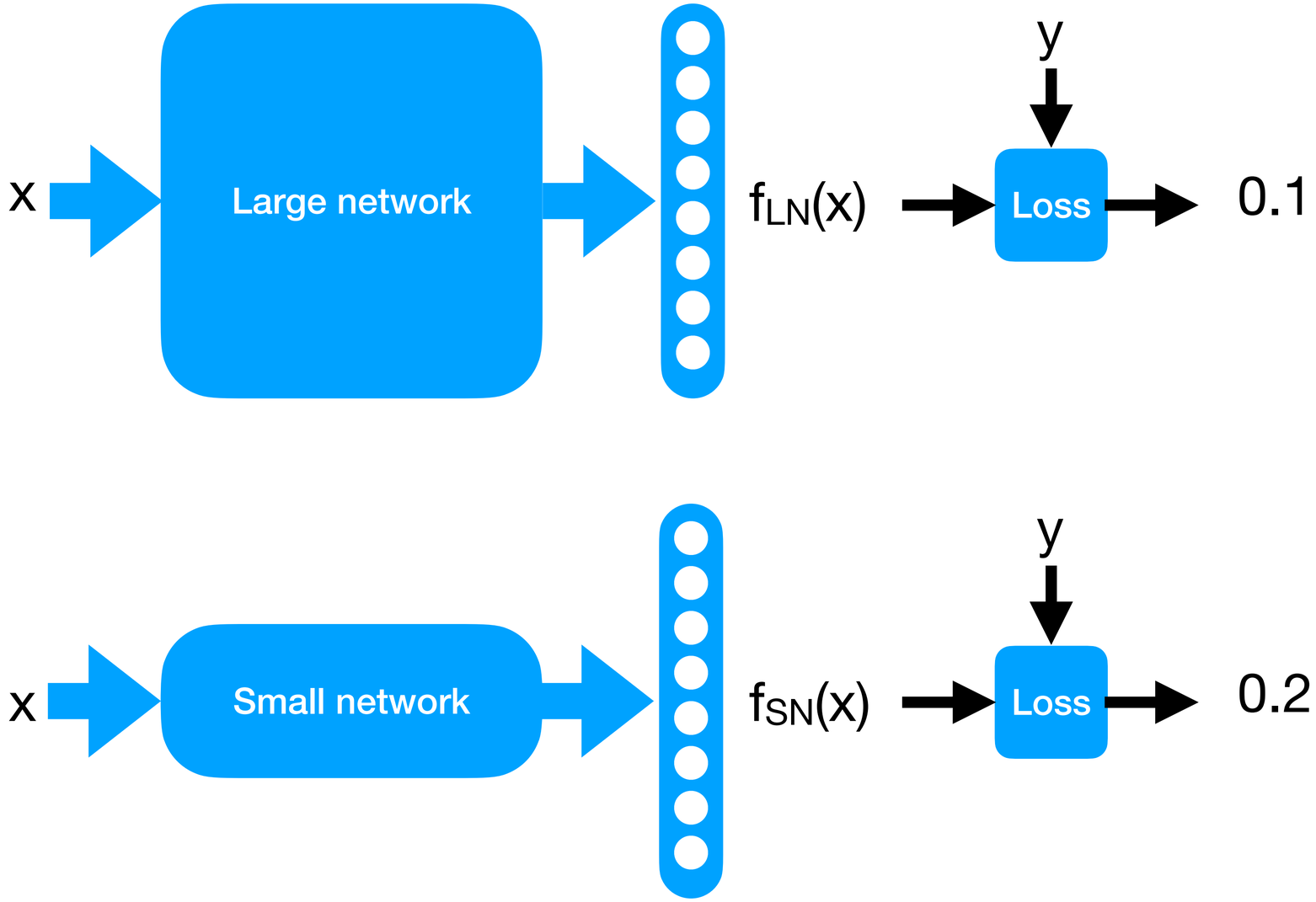
Knowledge distillation



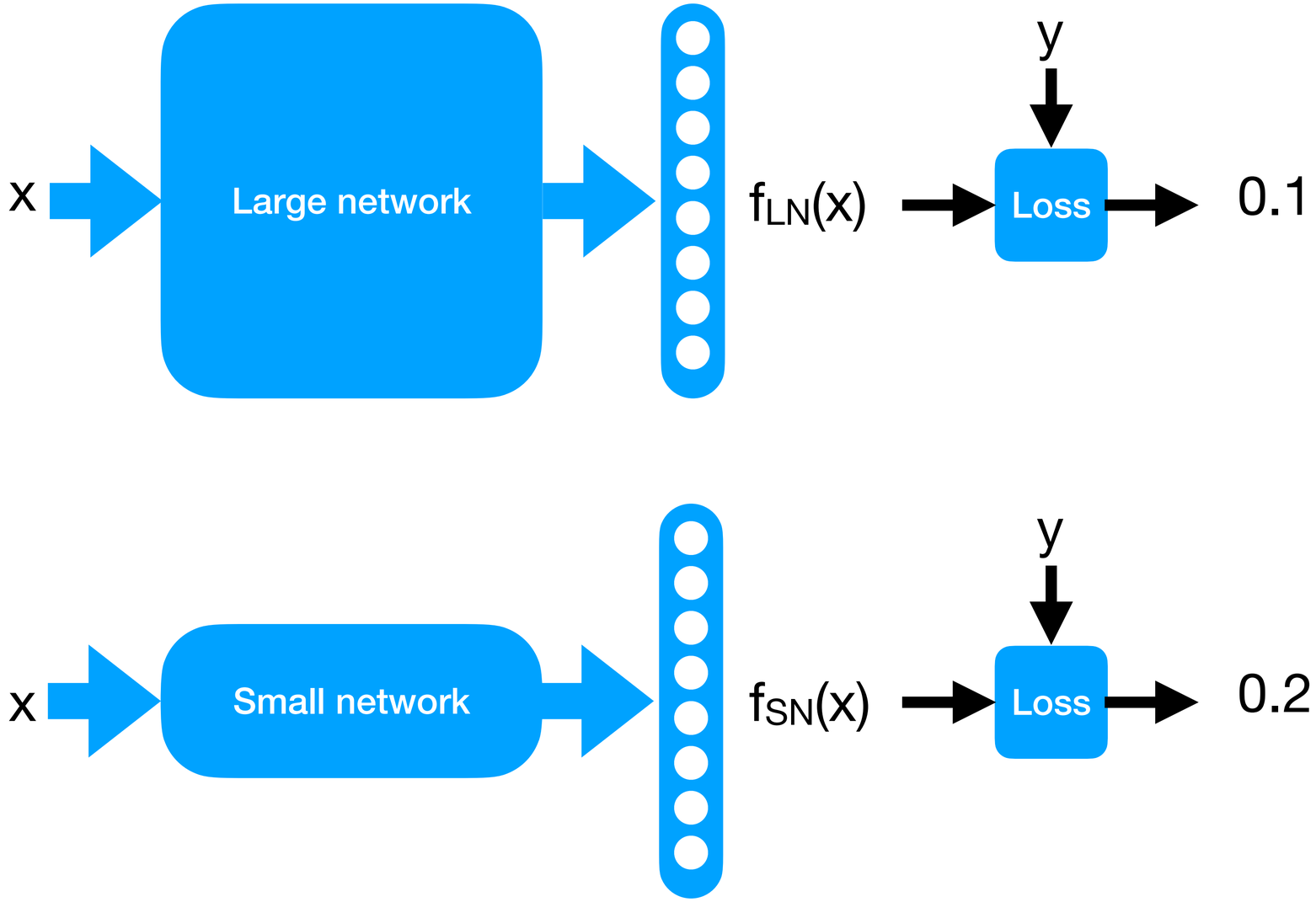
Knowledge distillation



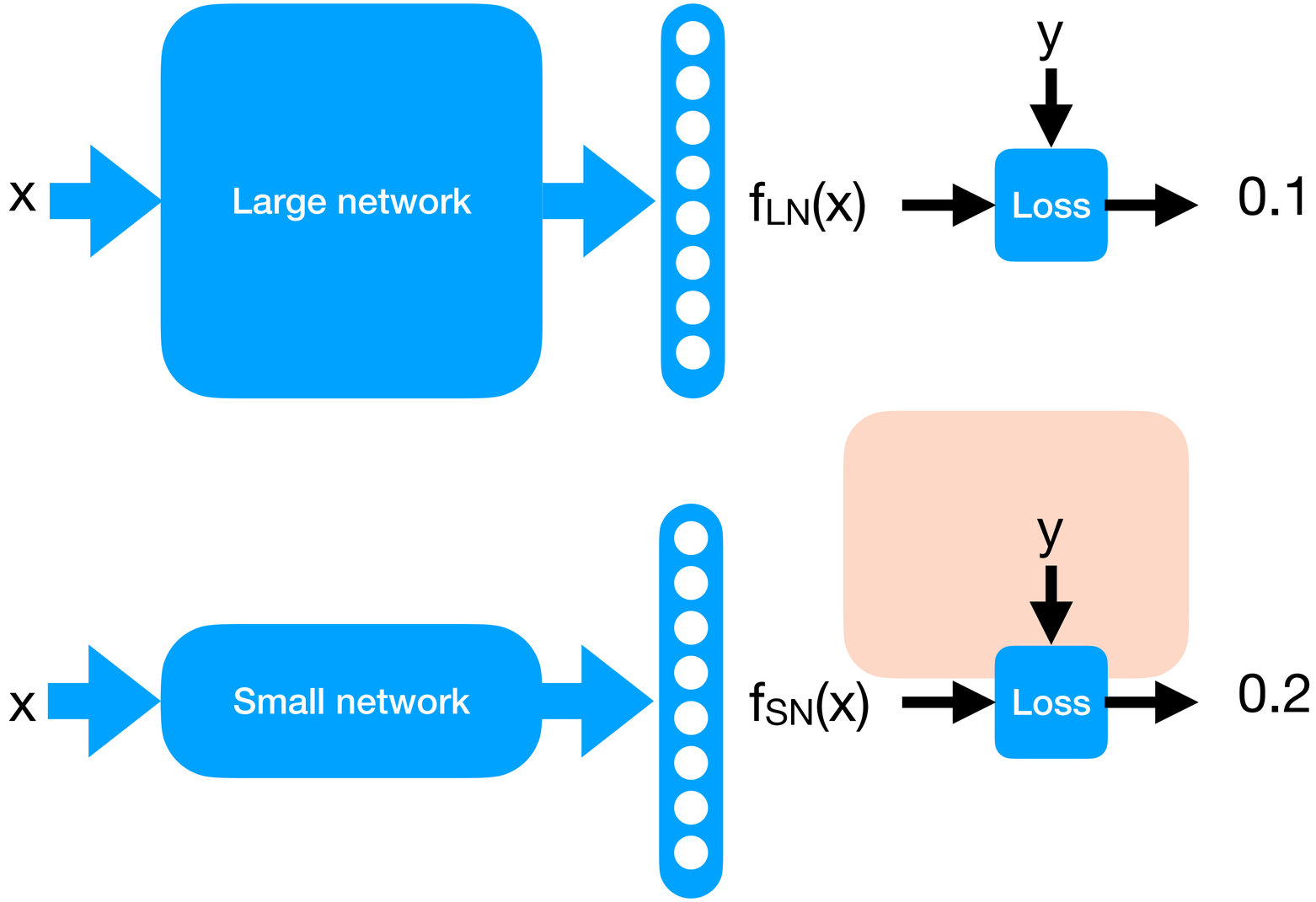
Knowledge distillation



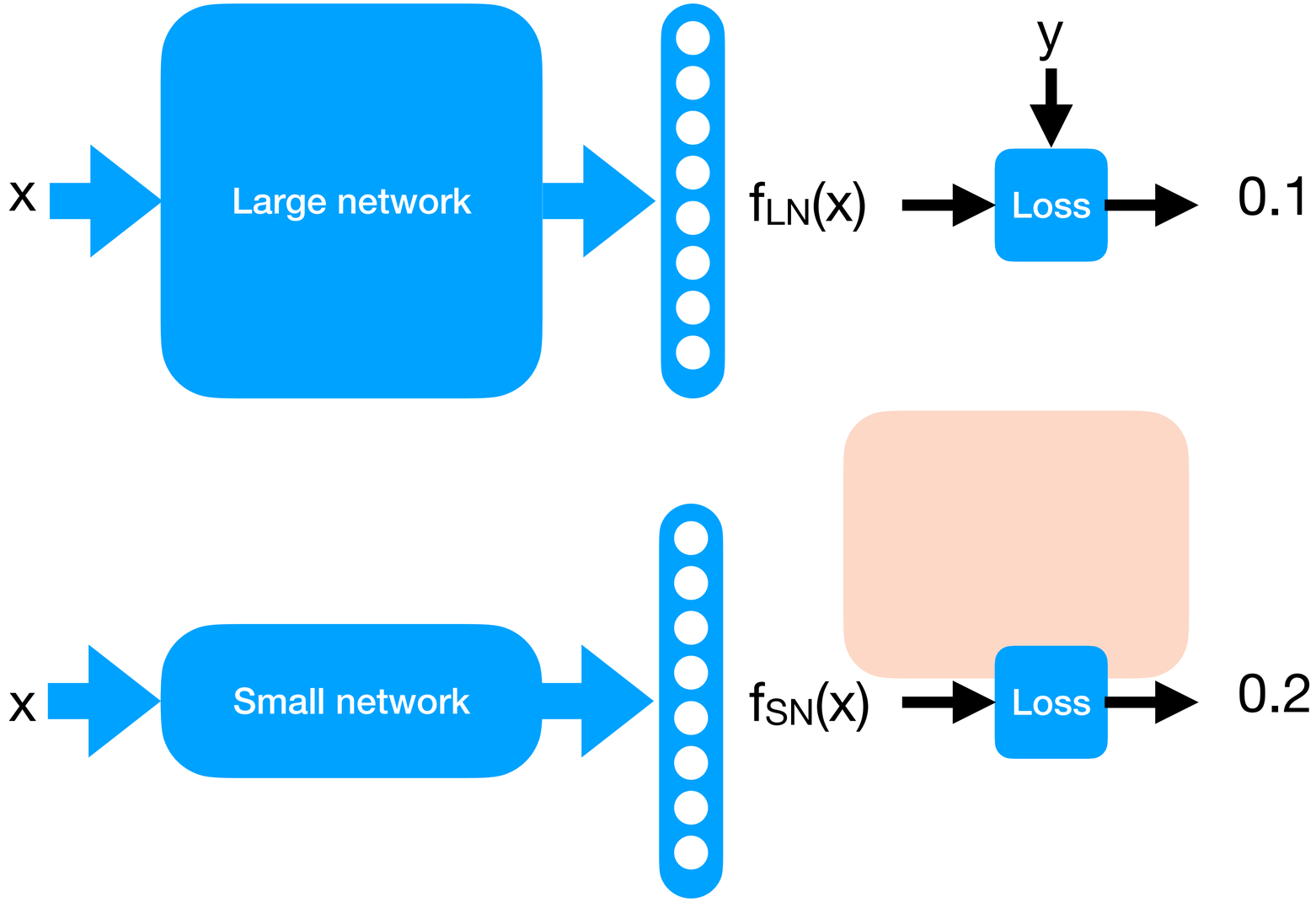
Knowledge distillation



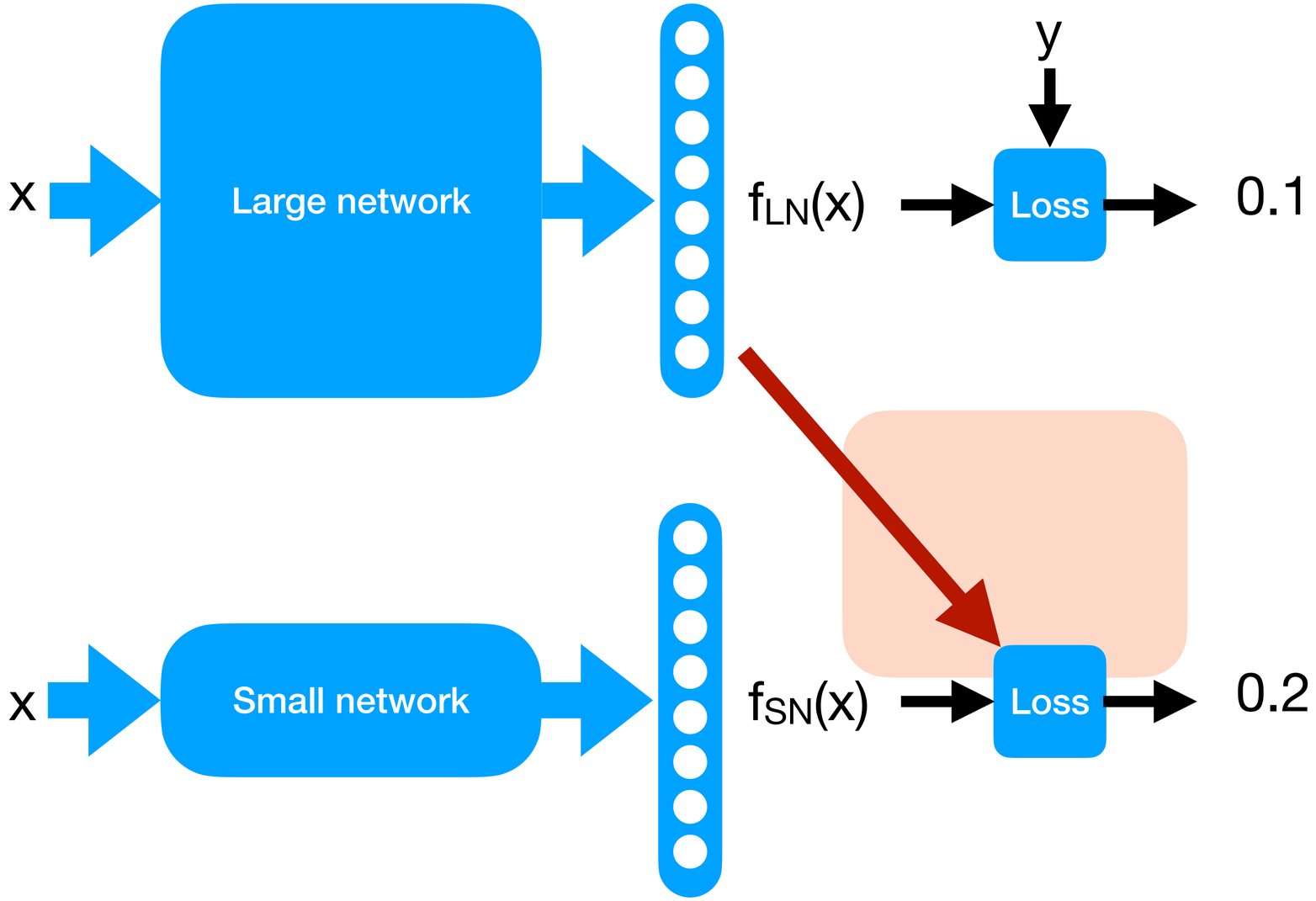
Knowledge distillation



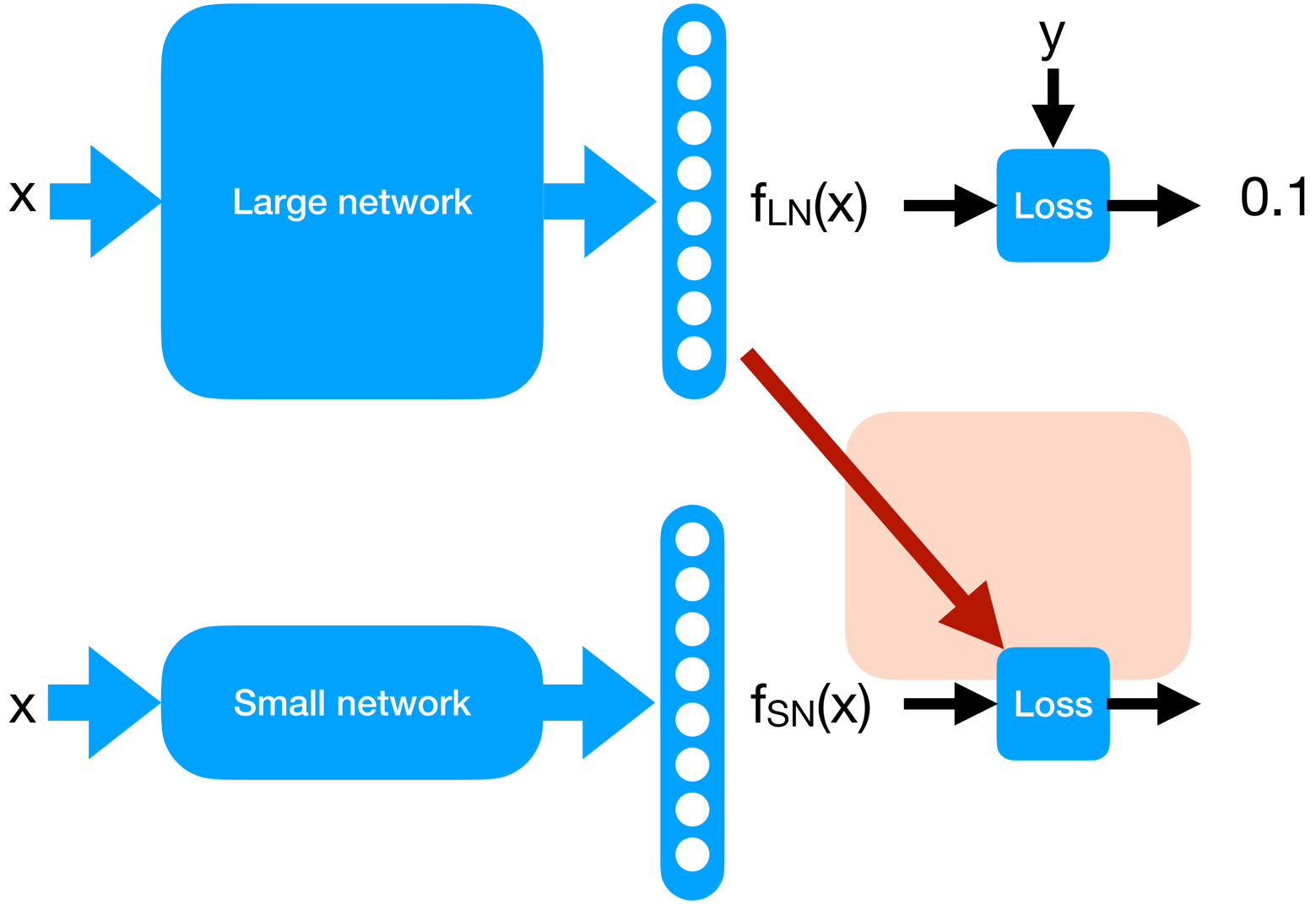
Knowledge distillation



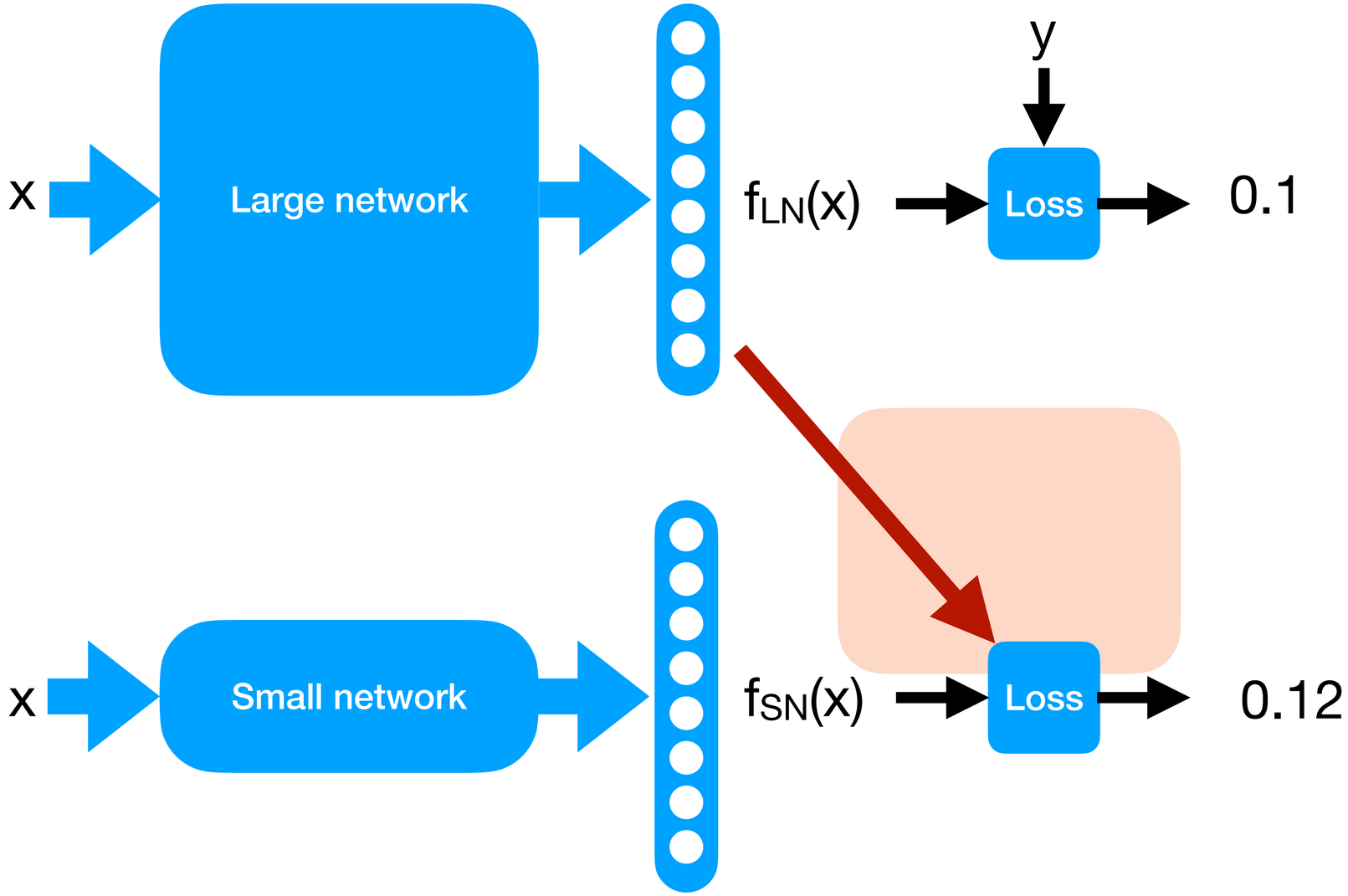
Knowledge distillation



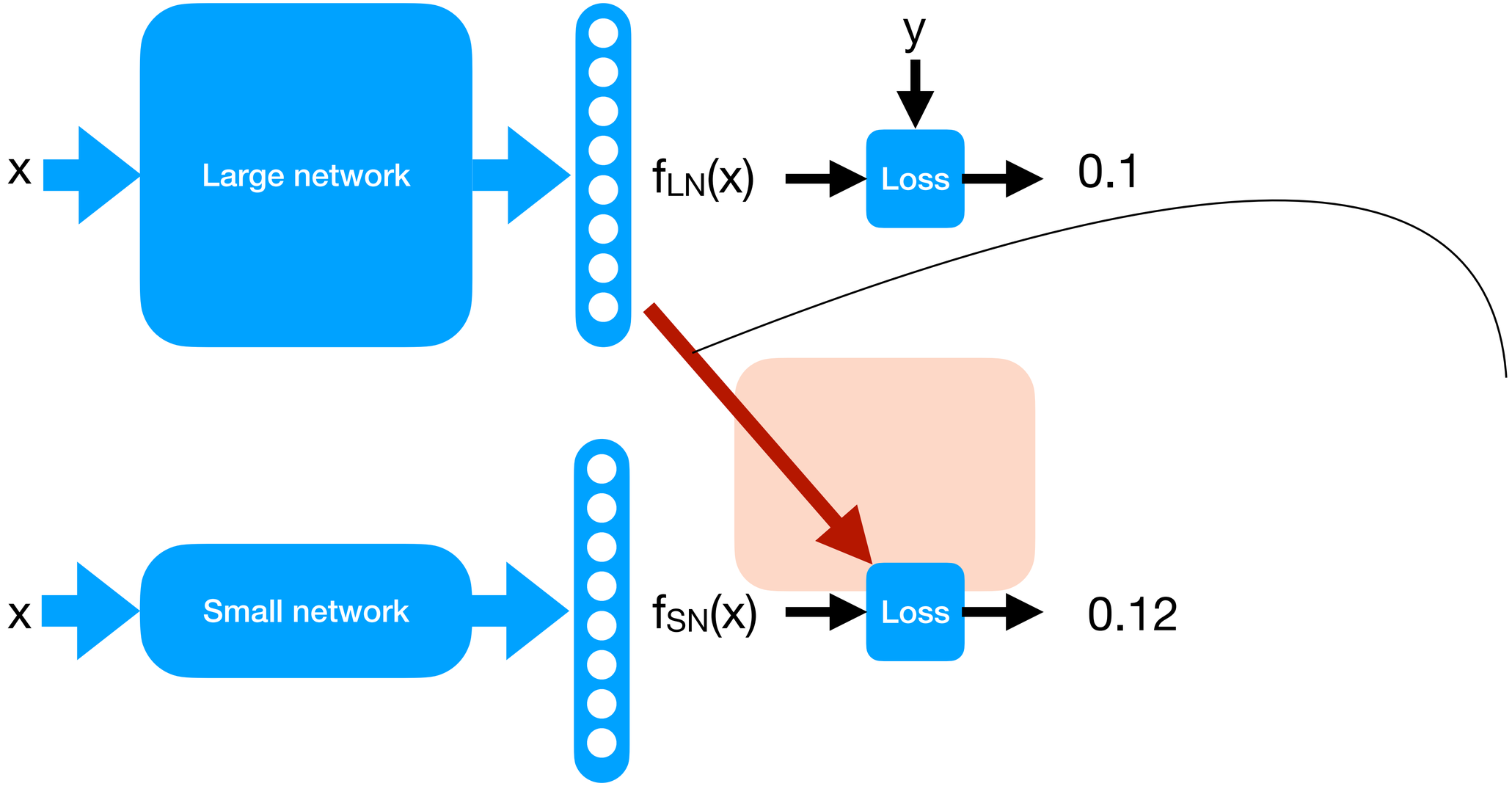
Knowledge distillation



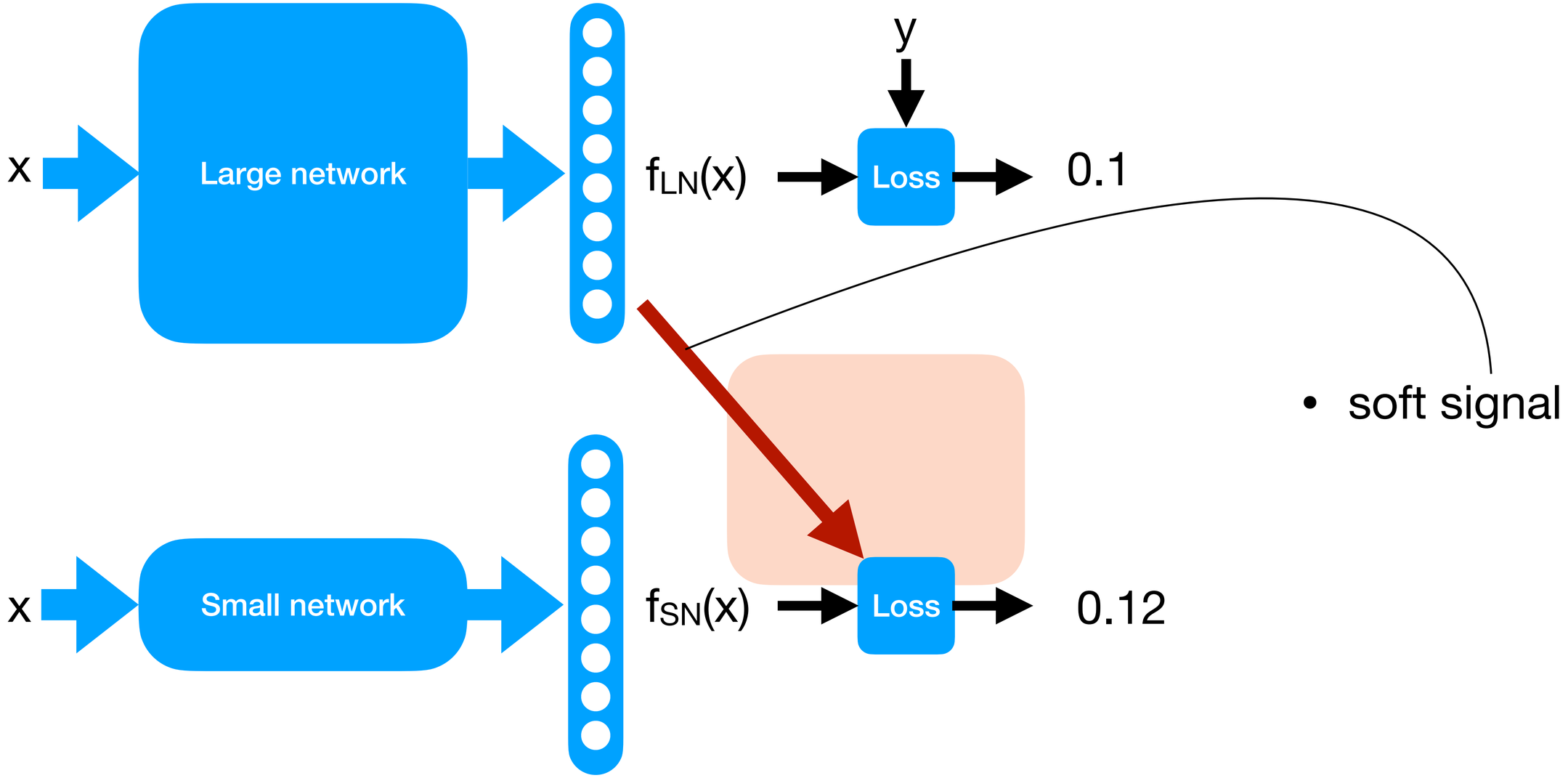
Knowledge distillation



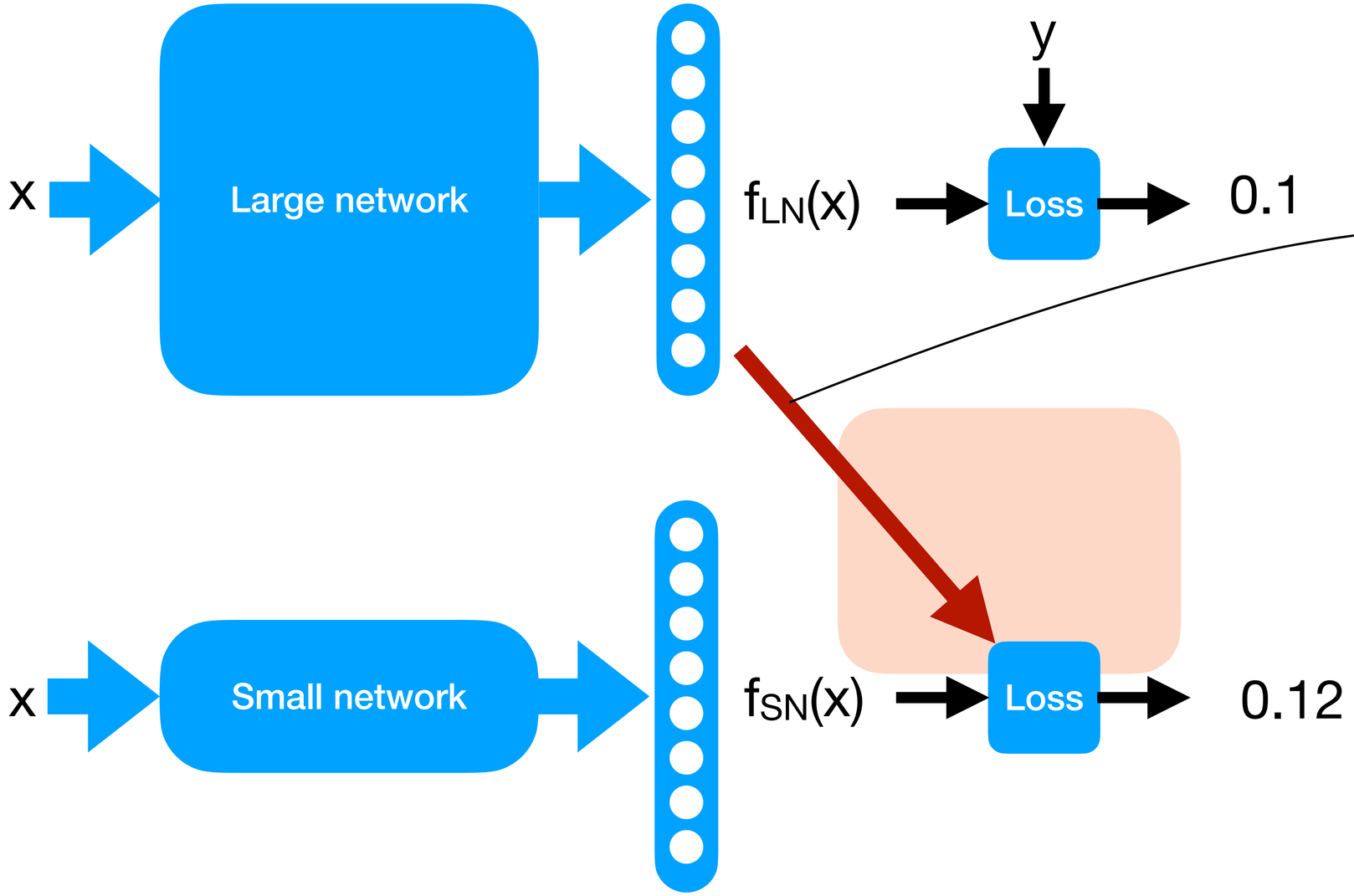
Knowledge distillation



Knowledge distillation

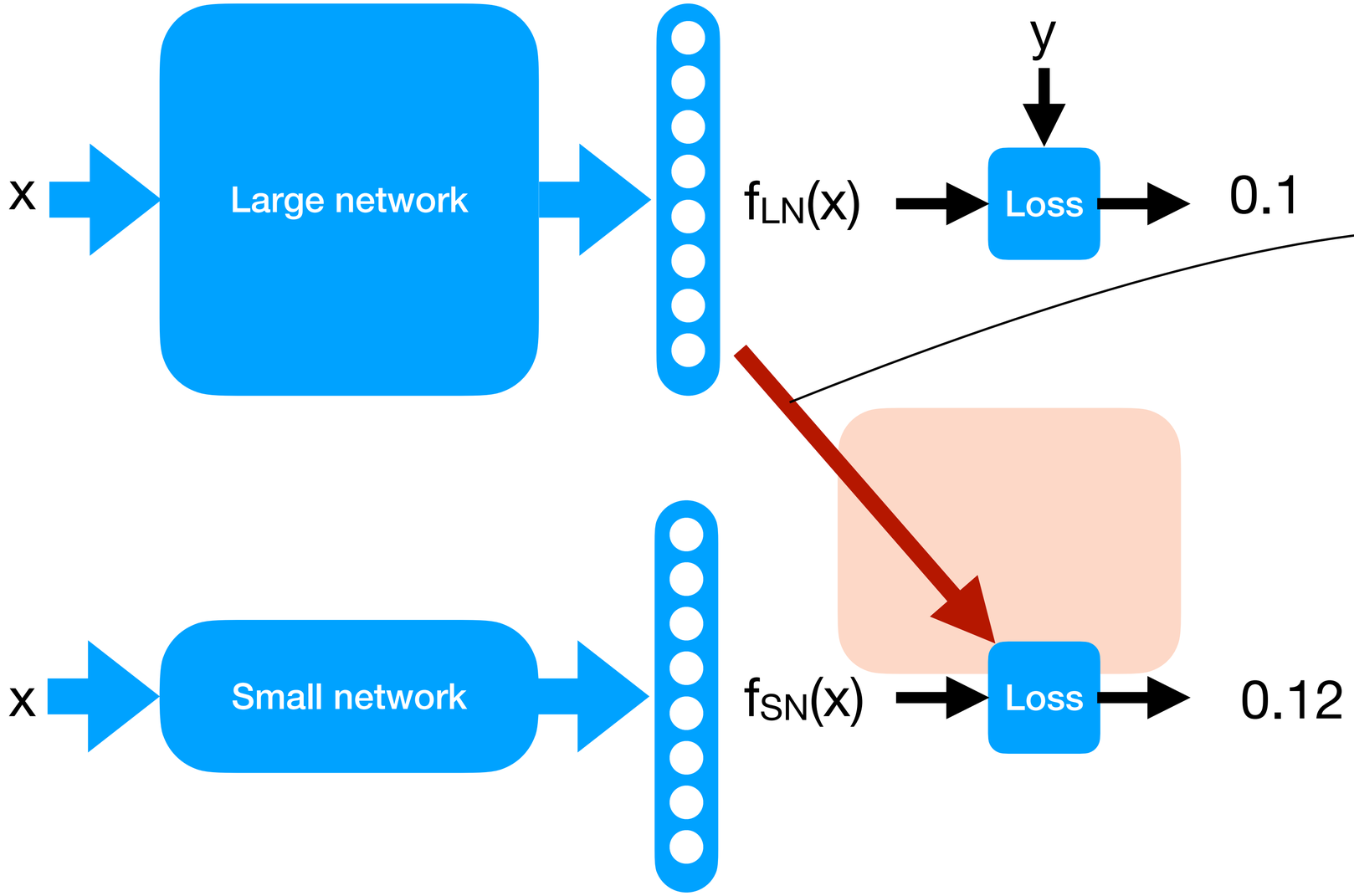


Knowledge distillation



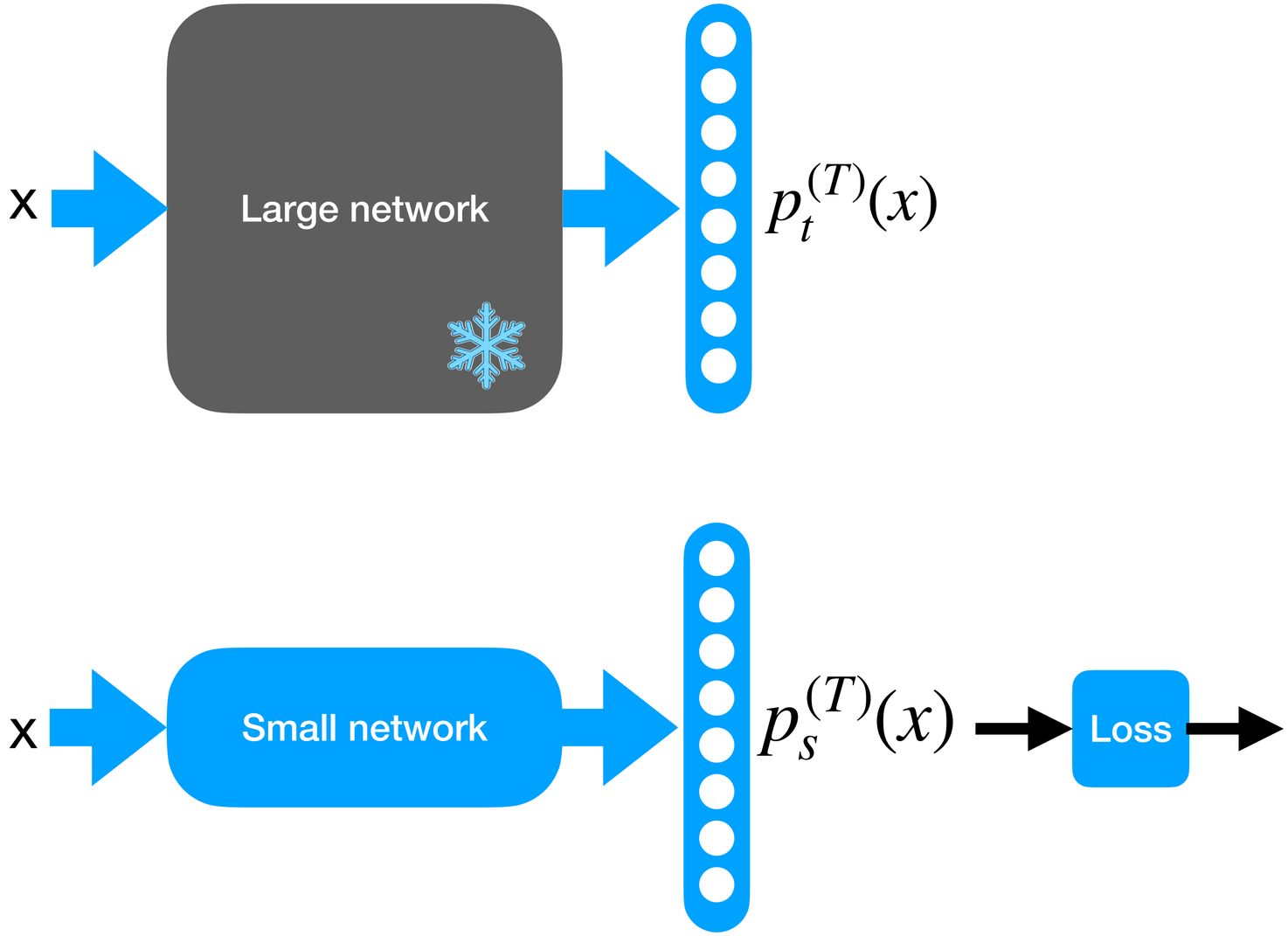
- soft signal
- easier to mimic

Knowledge distillation

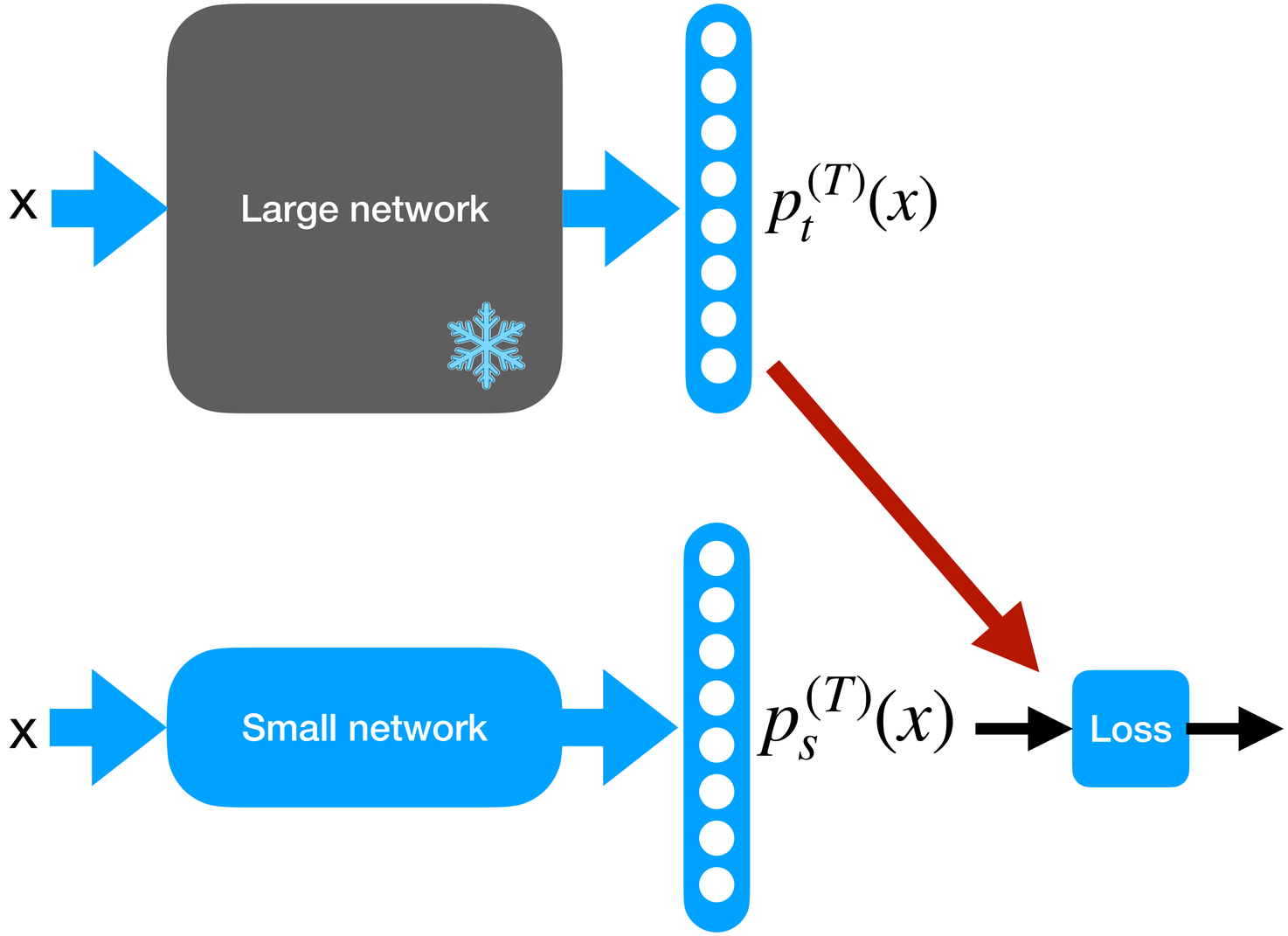


- soft signal
- easier to mimic
- Tough cases gone

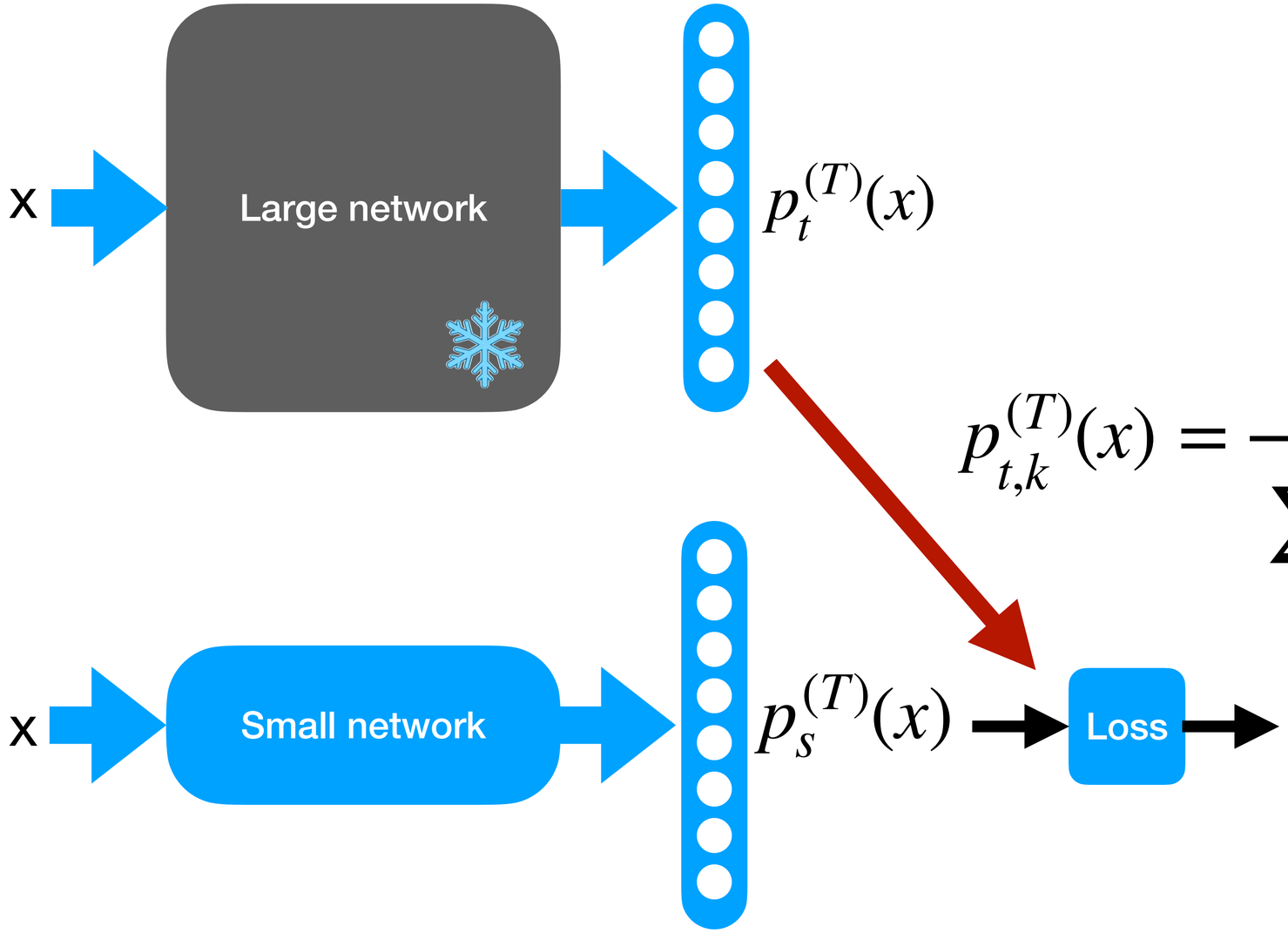
Knowledge distillation



Knowledge distillation

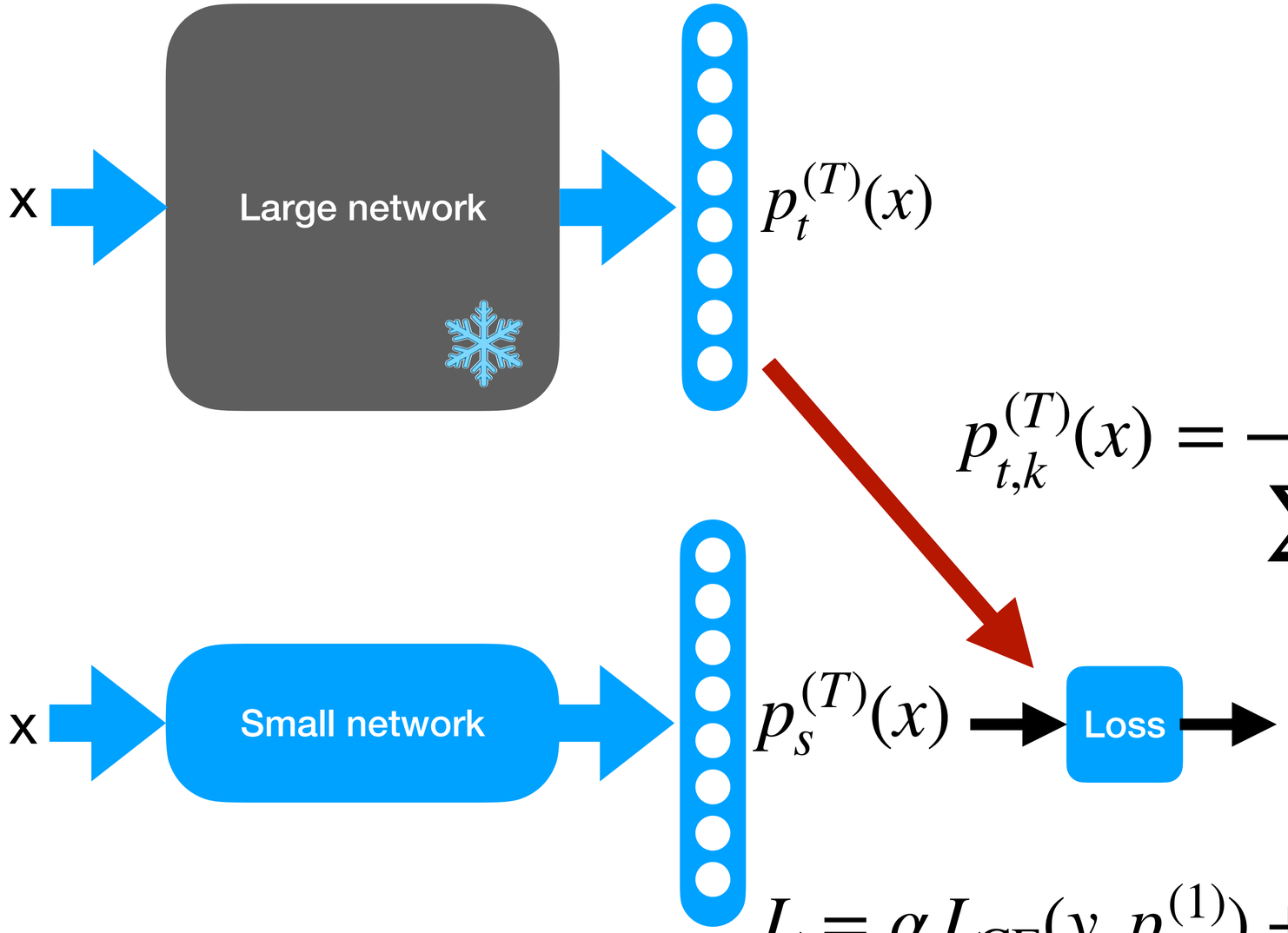


Knowledge distillation



$$p_{t,k}^{(T)}(x) = \frac{\exp\left(z_{t,k}(x)/T\right)}{\sum_{j=1}^{K} \exp\left(z_{t,j}(x)/T\right)}$$

Knowledge distillation



$$p_{t,k}^{(T)}(x) = \frac{\exp\left(z_{t,k}(x)/T\right)}{\sum_{j=1}^{K} \exp\left(z_{t,j}(x)/T\right)}$$

$$L = \alpha\, L_{\mathrm{CE}}(y, p_s^{(1)}) + (1-\alpha)\, T^2\, \mathrm{KL}\left(p_t^{(T)} \,\|\, p_s^{(T)}\right)$$

Distillation with no labels?

Self-supervised distillation

Emerging Properties in Self-Supervised Vision Transformers

Mathilde Caron[1,2] Hugo Touvron[1,3] Ishan Misra[1] Hervé Jegou[1]
Julien Mairal[2] Piotr Bojanowski[1] Armand Joulin[1]

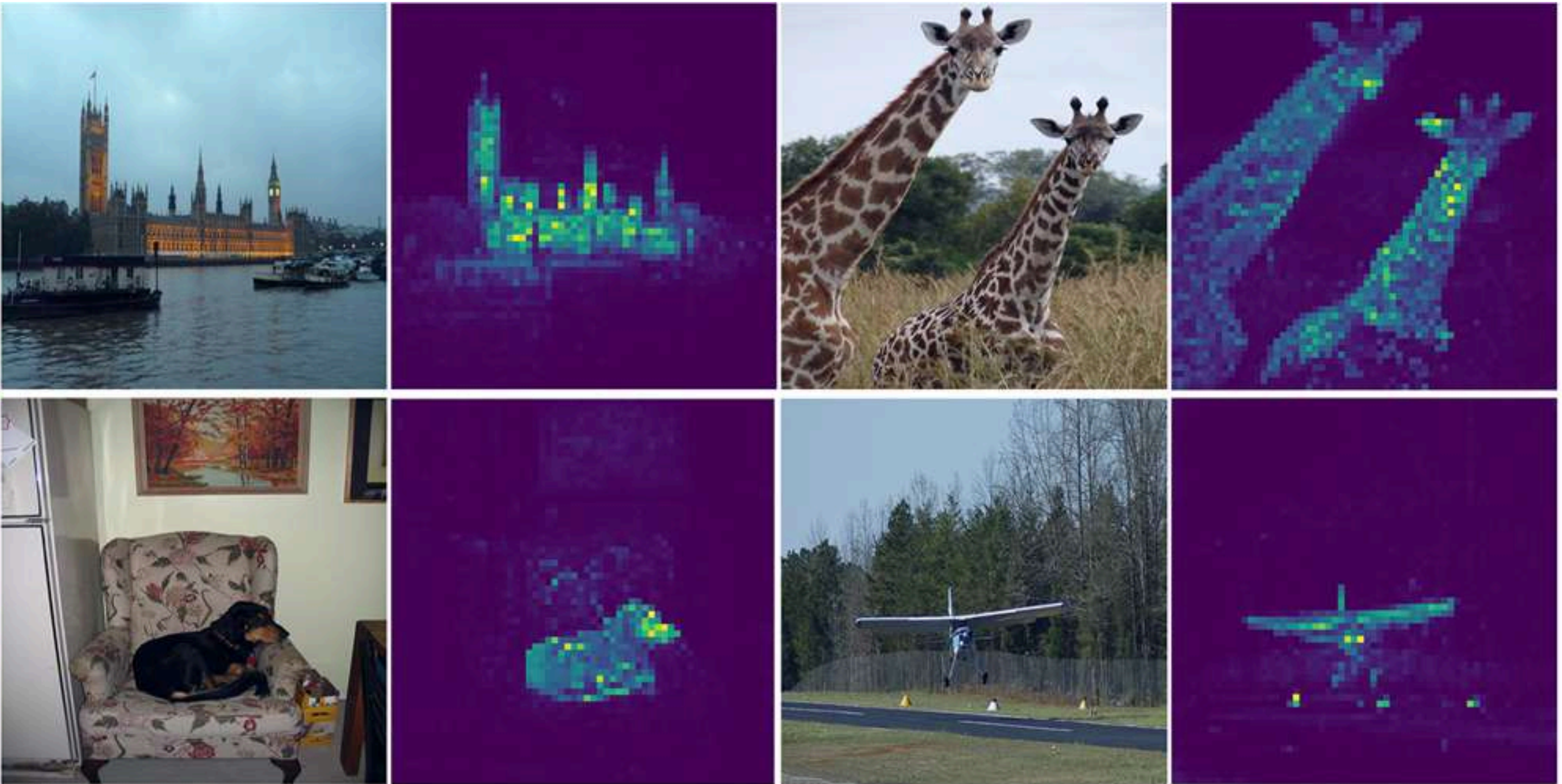
[1] Facebook AI Research [2] Inria* [3] Sorbonne University

[NeurIPS2021]

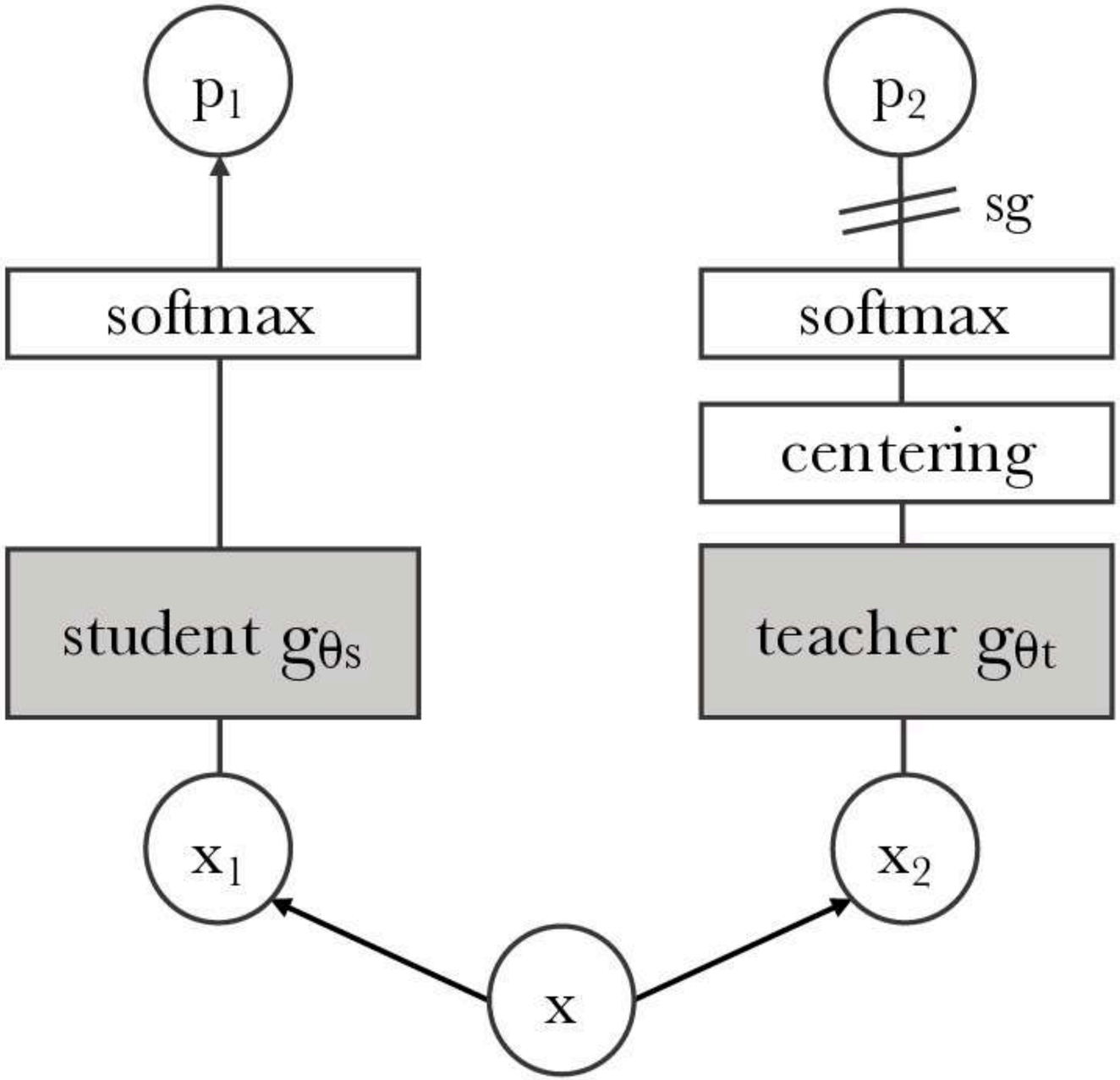
DINO = self **DI**stillation with **NO** labels

Claim: with no training supervision self-attention discovers objects

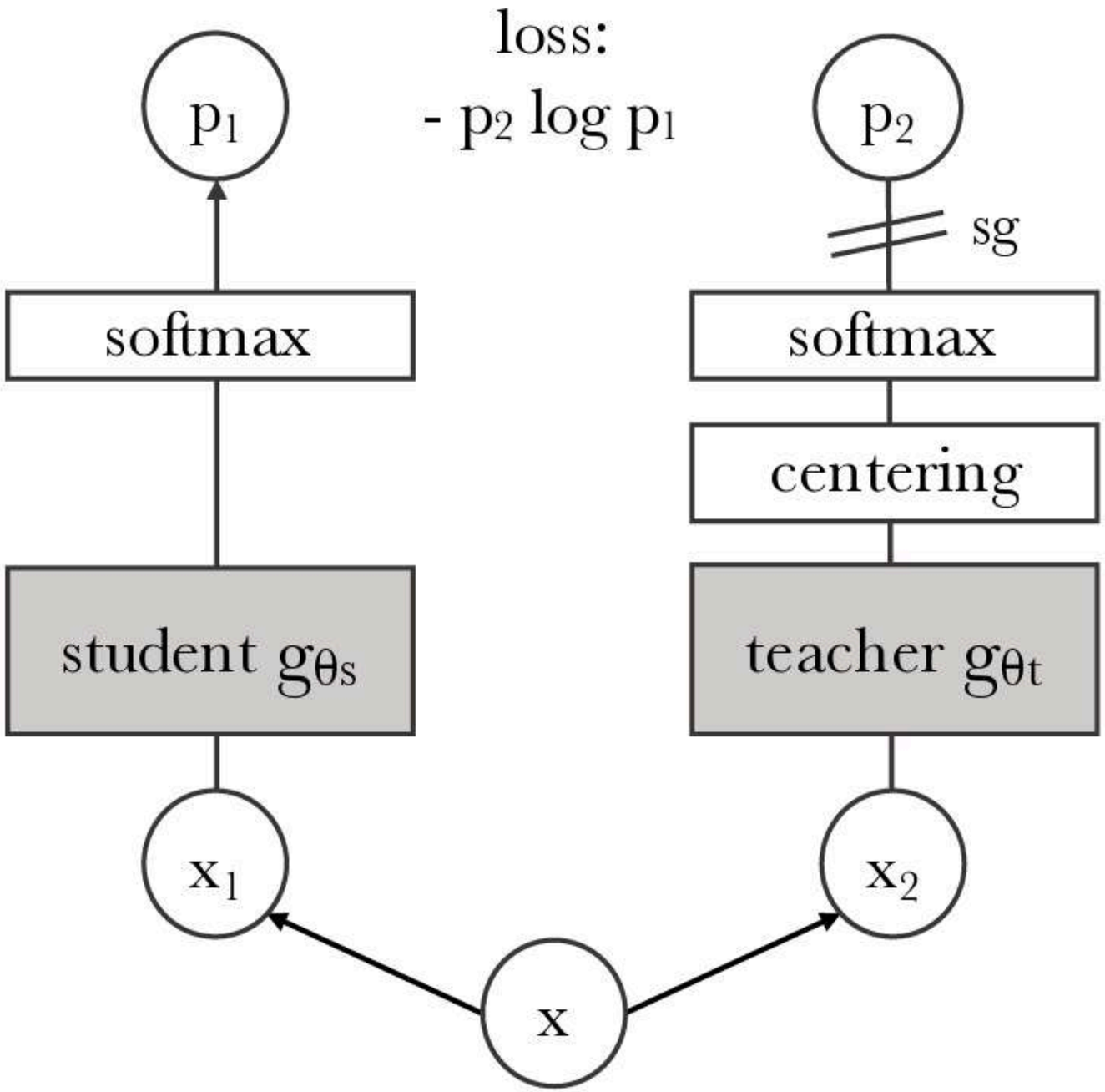
Claim: with no training supervision self-attention discovers objects



Student-teacher arrangement



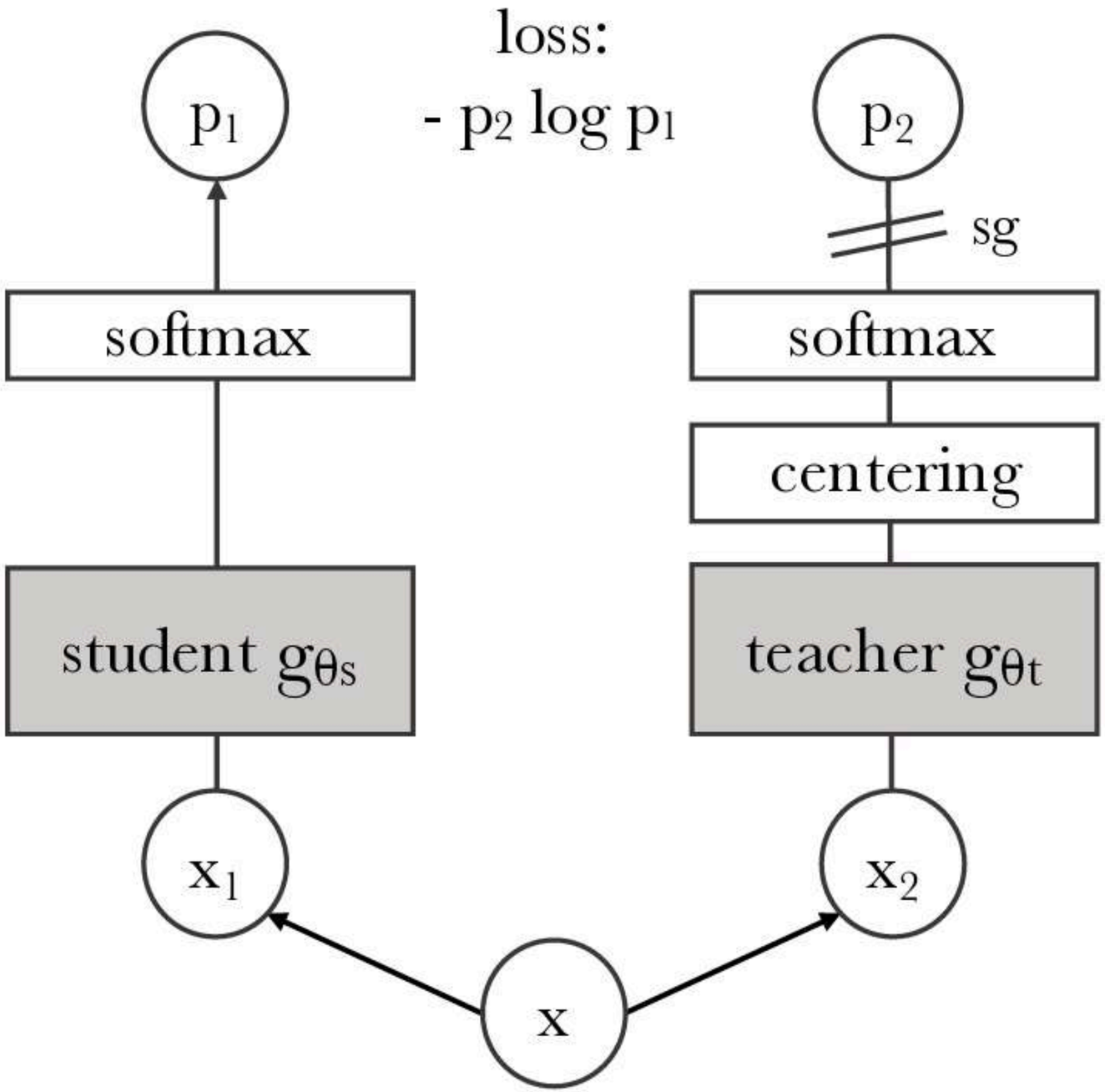
Student training



Student training

Temperature-dependent softmax:

$$P_s^{(i)}(x) = \frac{\exp\left(g_{\theta_s}^{(i)}(x)/\tau_s\right)}{\sum_{k=1}^{K} \exp\left(g_{\theta_s}^{(k)}(x)/\tau_s\right)}$$



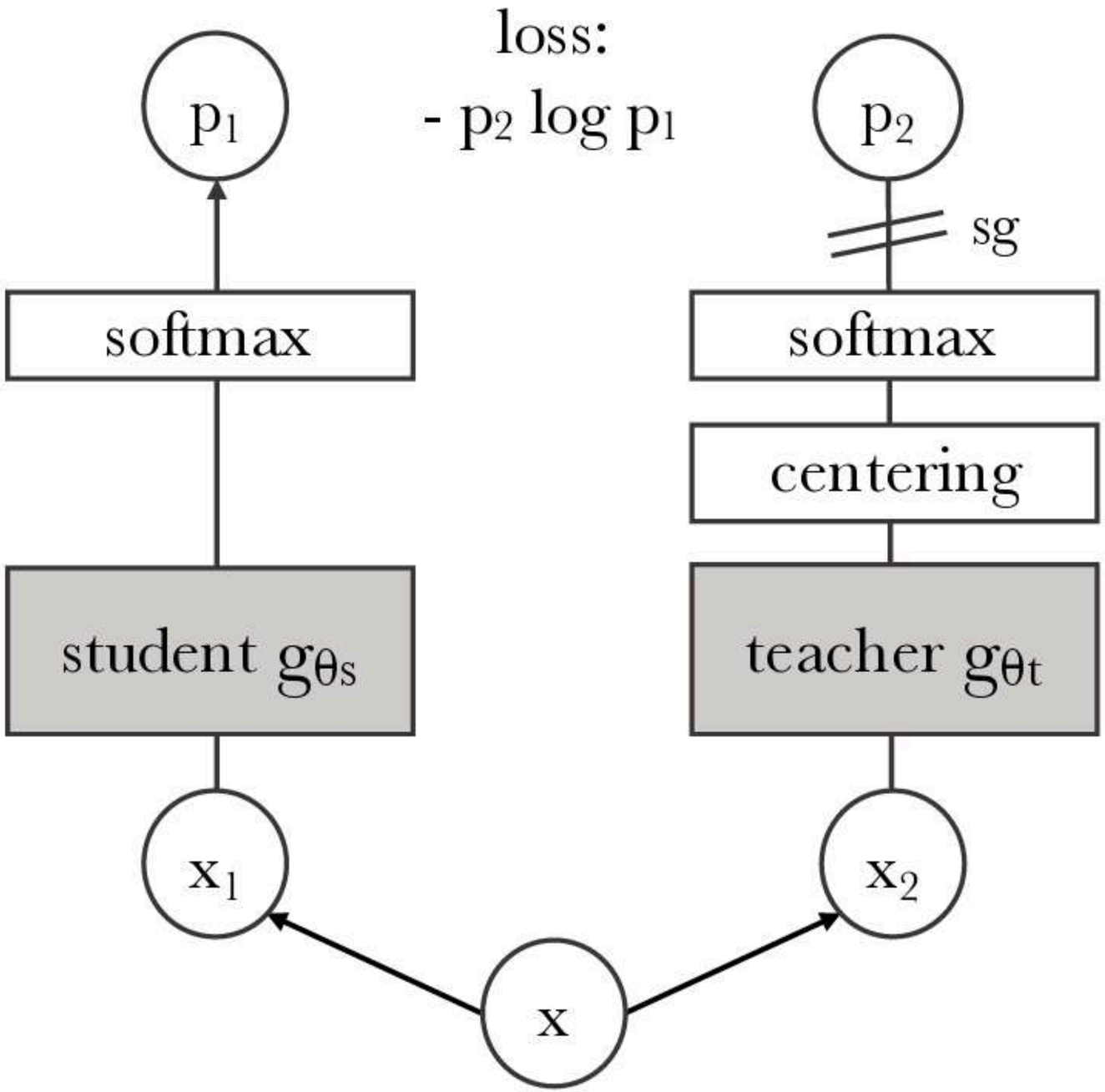
Student training

Temperature-dependent softmax:

$$P_s^{(i)}(x) = \frac{\exp\left(g_{\theta_s}^{(i)}(x)/\tau_s\right)}{\sum_{k=1}^{K} \exp\left(g_{\theta_s}^{(k)}(x)/\tau_s\right)}$$

Loss for student network:

$$\min_{\theta_s} \; H\left(P_t(x), P_s(x)\right)$$



Student training

Temperature-dependent softmax:

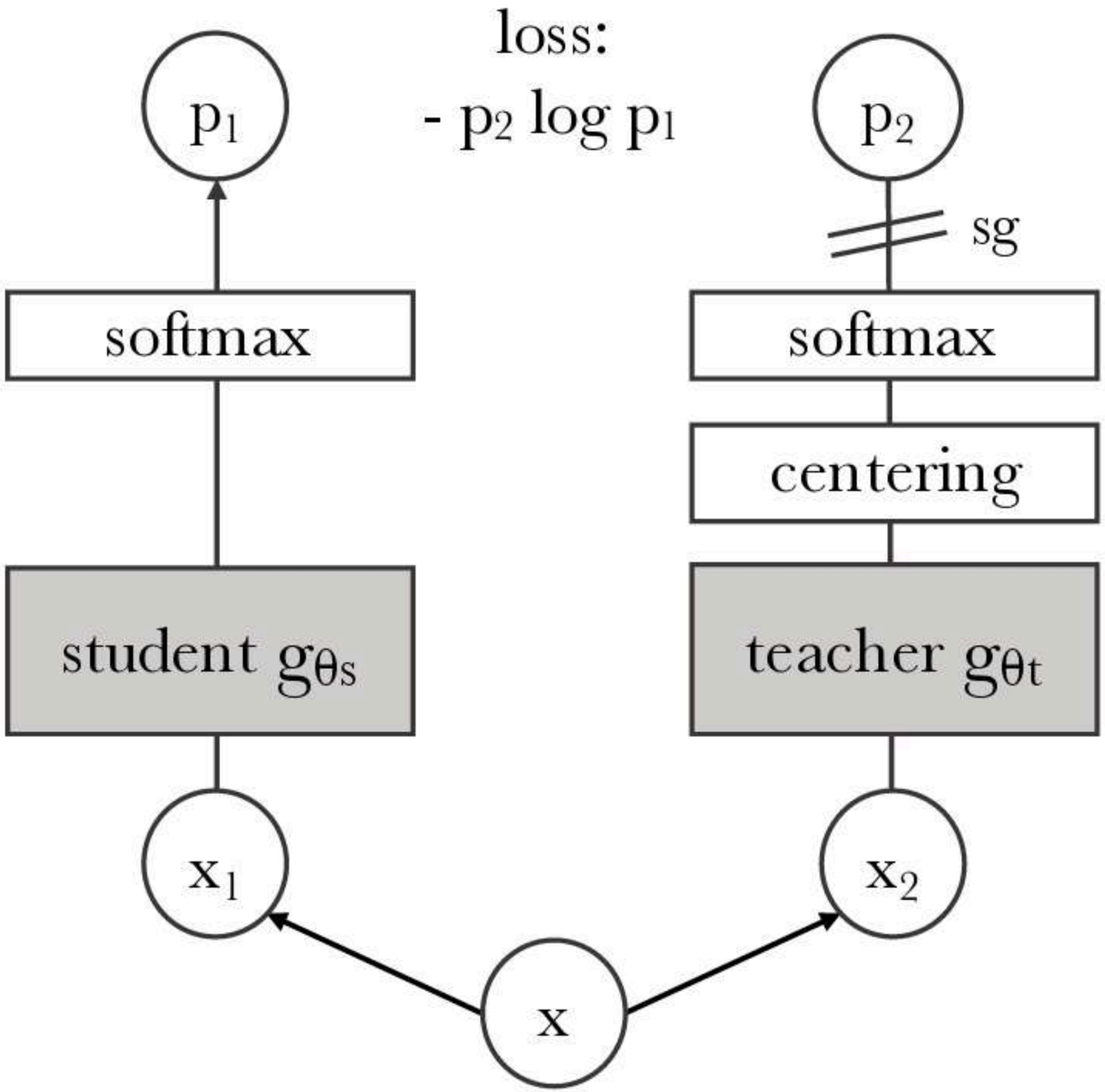
$$P_s^{(i)}(x) = \frac{\exp\left(g_{\theta_s}^{(i)}(x)/\tau_s\right)}{\sum_{k=1}^{K} \exp\left(g_{\theta_s}^{(k)}(x)/\tau_s\right)}$$

Loss for student network:

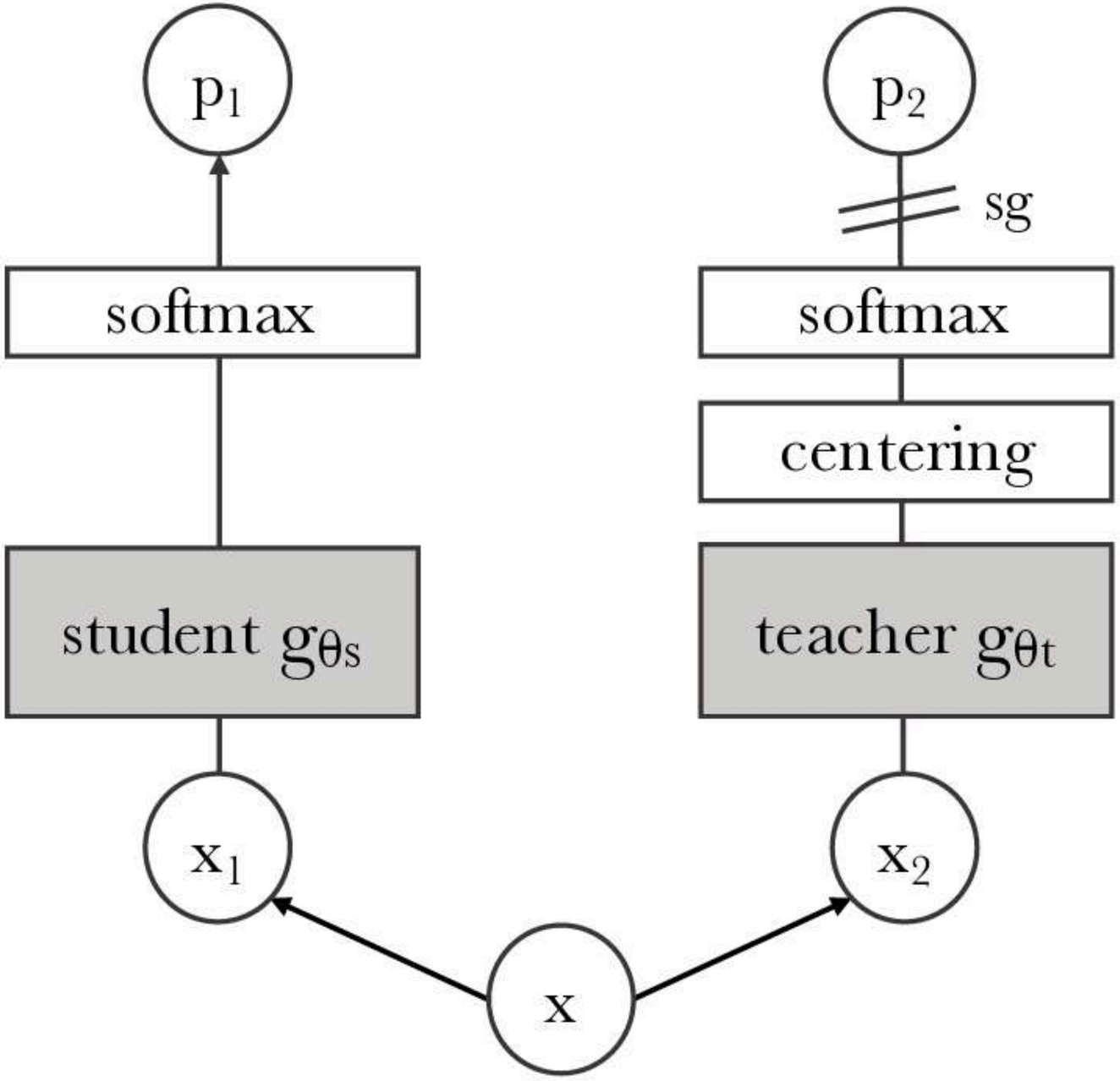
$$\min_{\theta_s} \; H\left(P_t(x), P_s(x)\right)$$

H is cross entropy:

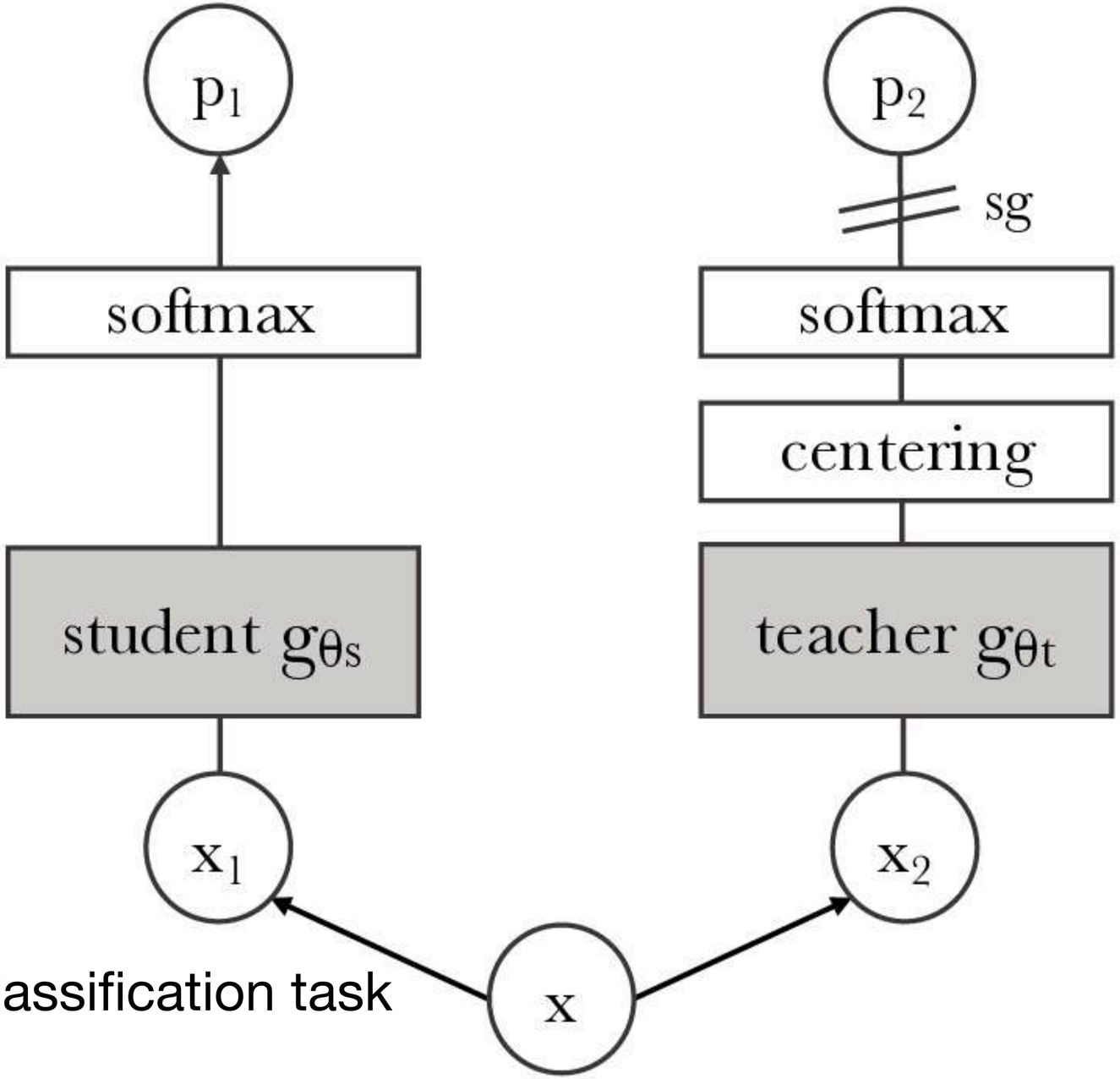
$$H(a,b) = -\sum_{i=1}^{K} a_i \log b_i$$



Who trains the teacher?

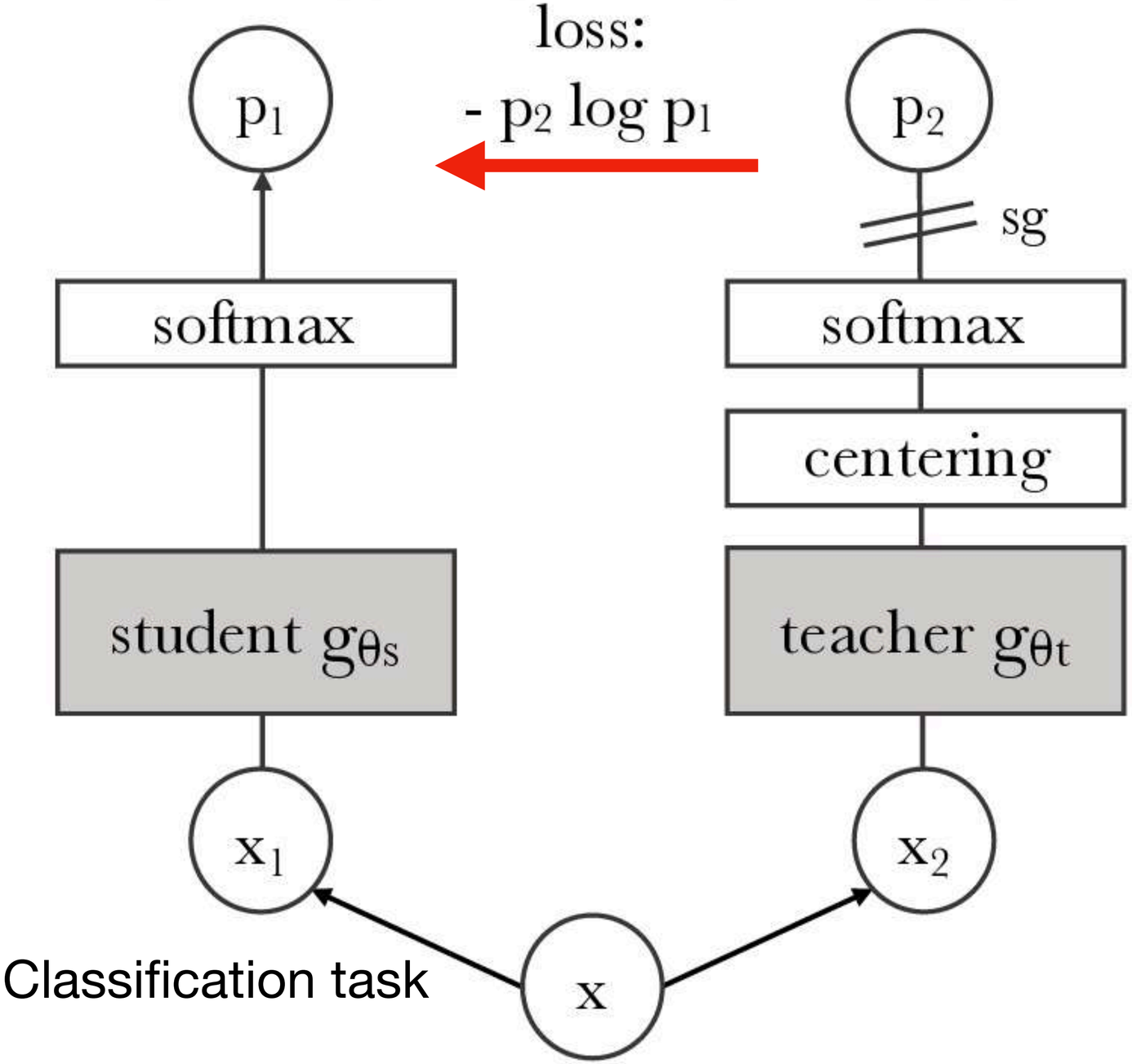


Who trains the teacher?



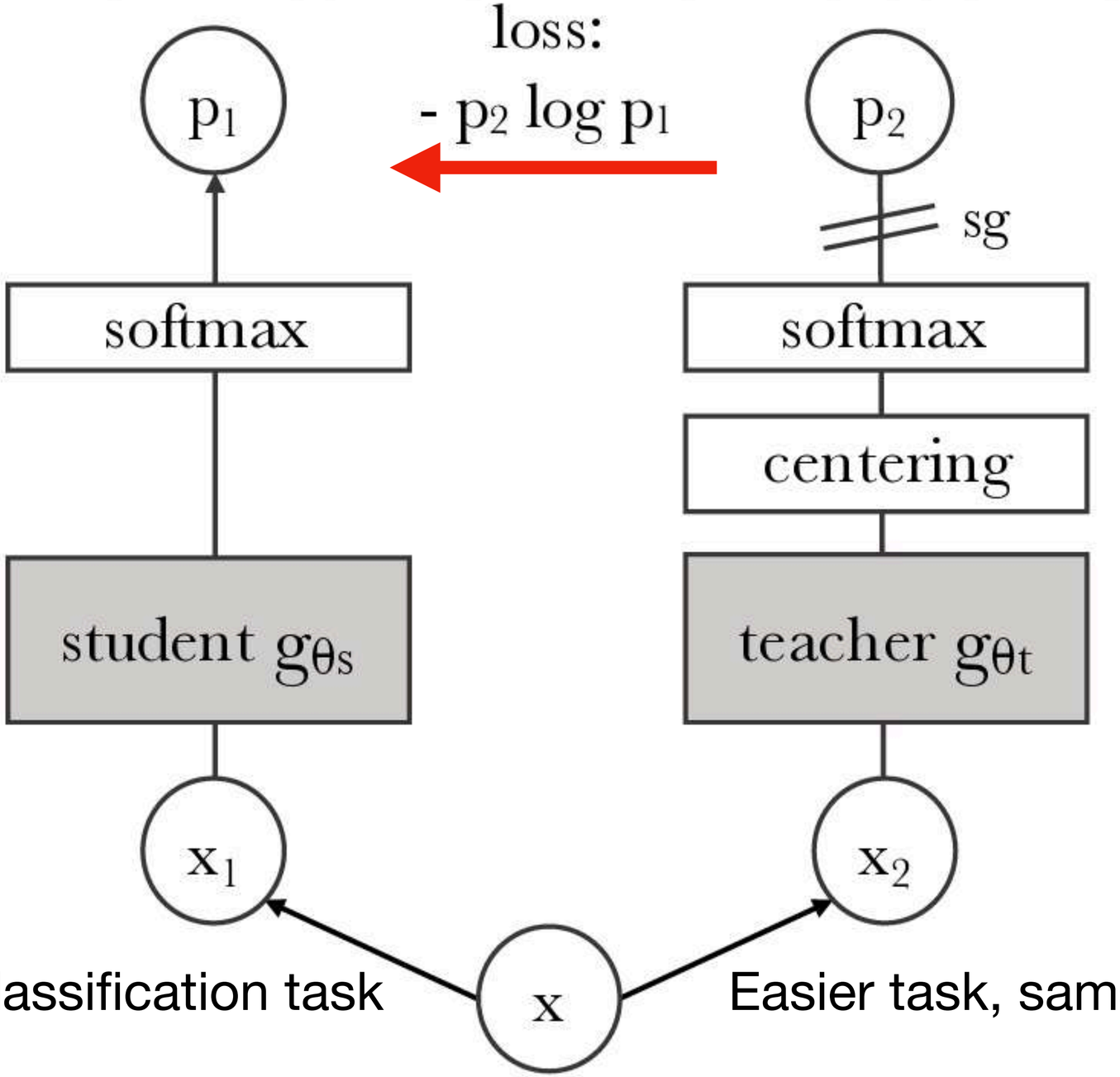
Classification task

Who trains the teacher?



Classification task

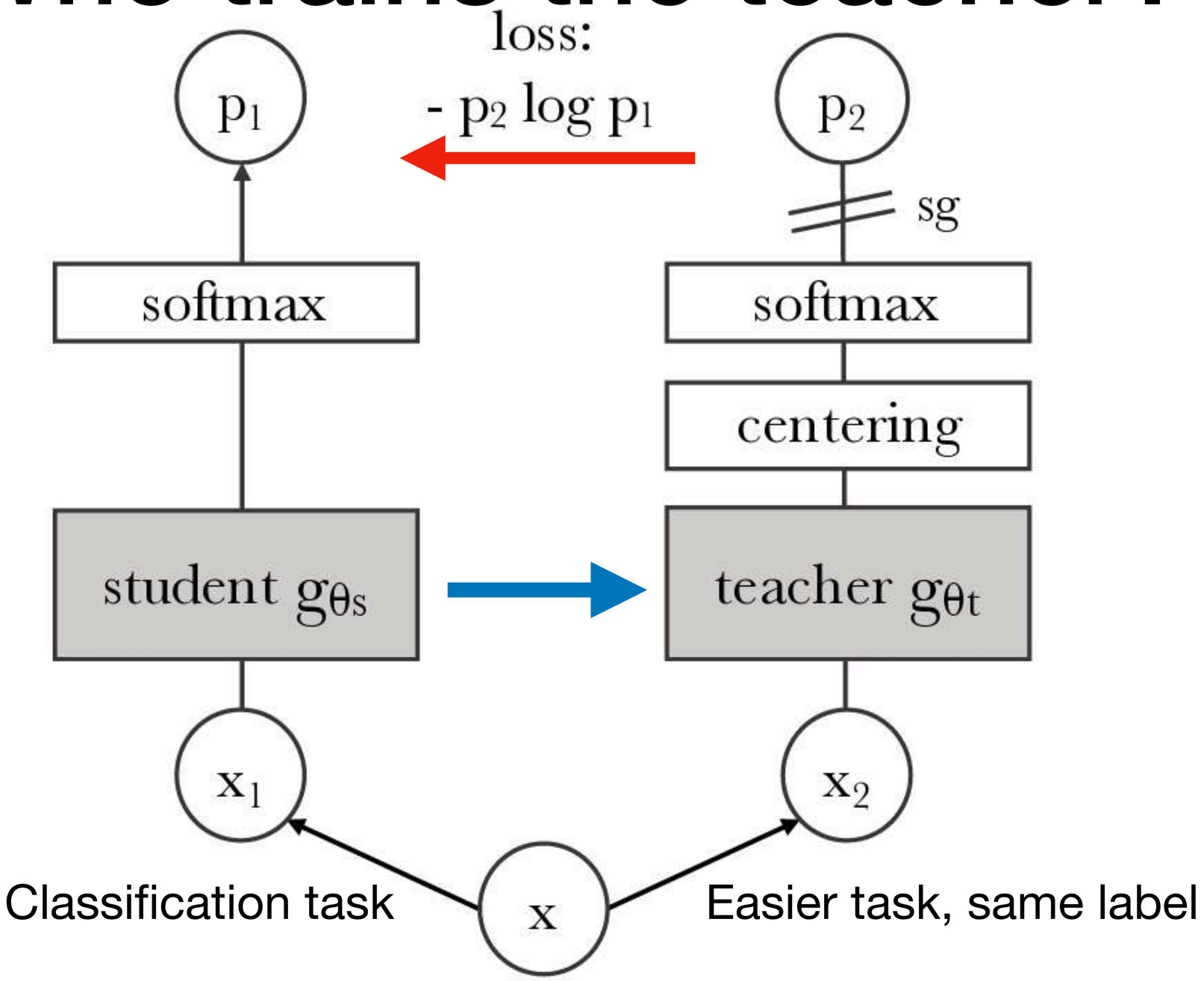
Who trains the teacher?



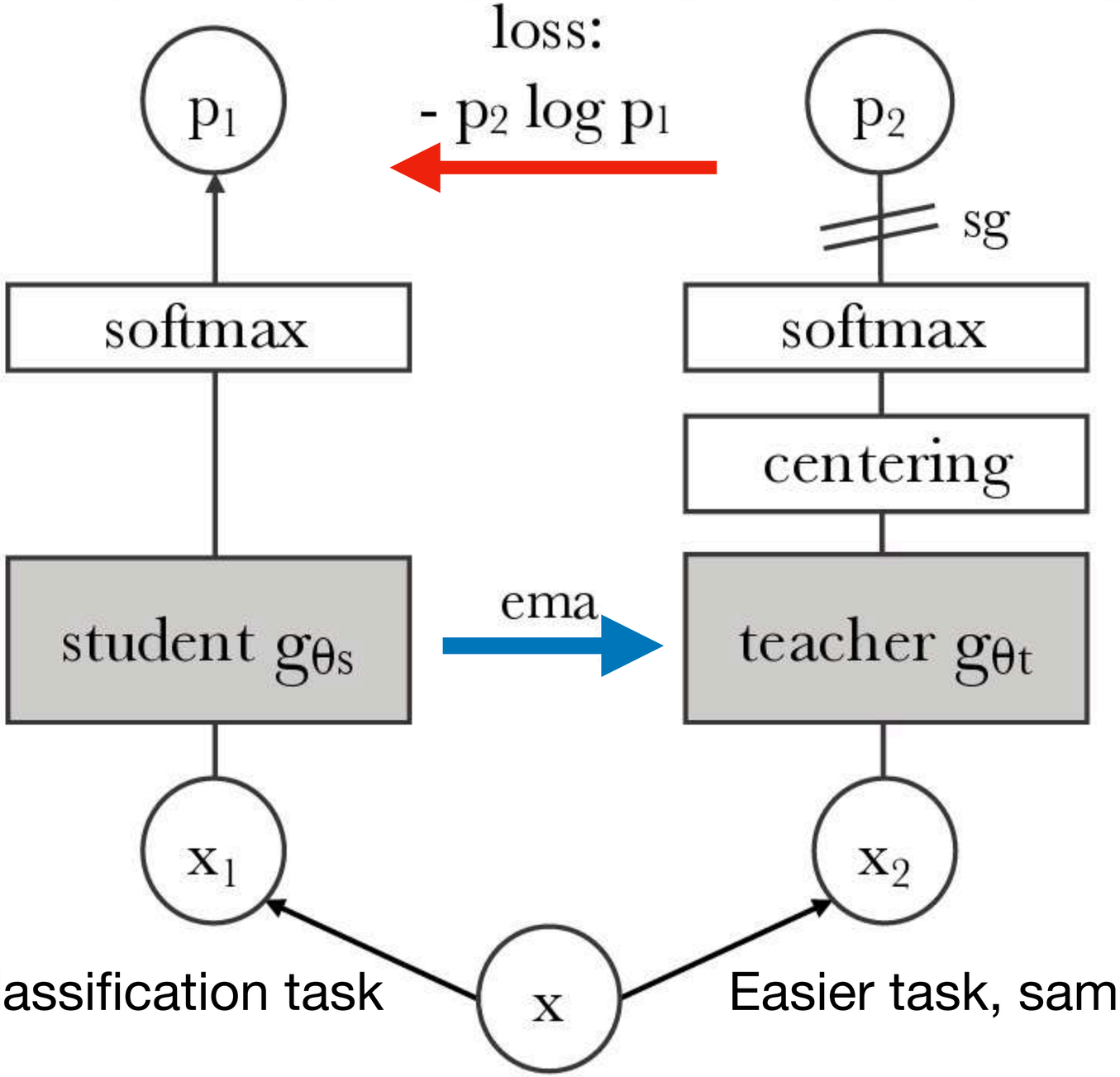
Classification task

Easier task, same label

Who trains the teacher?



Who trains the teacher?



Classification task

Easier task, same label

Exponential Moving Average (EMA)

[Figure from S. Readings, 2025]

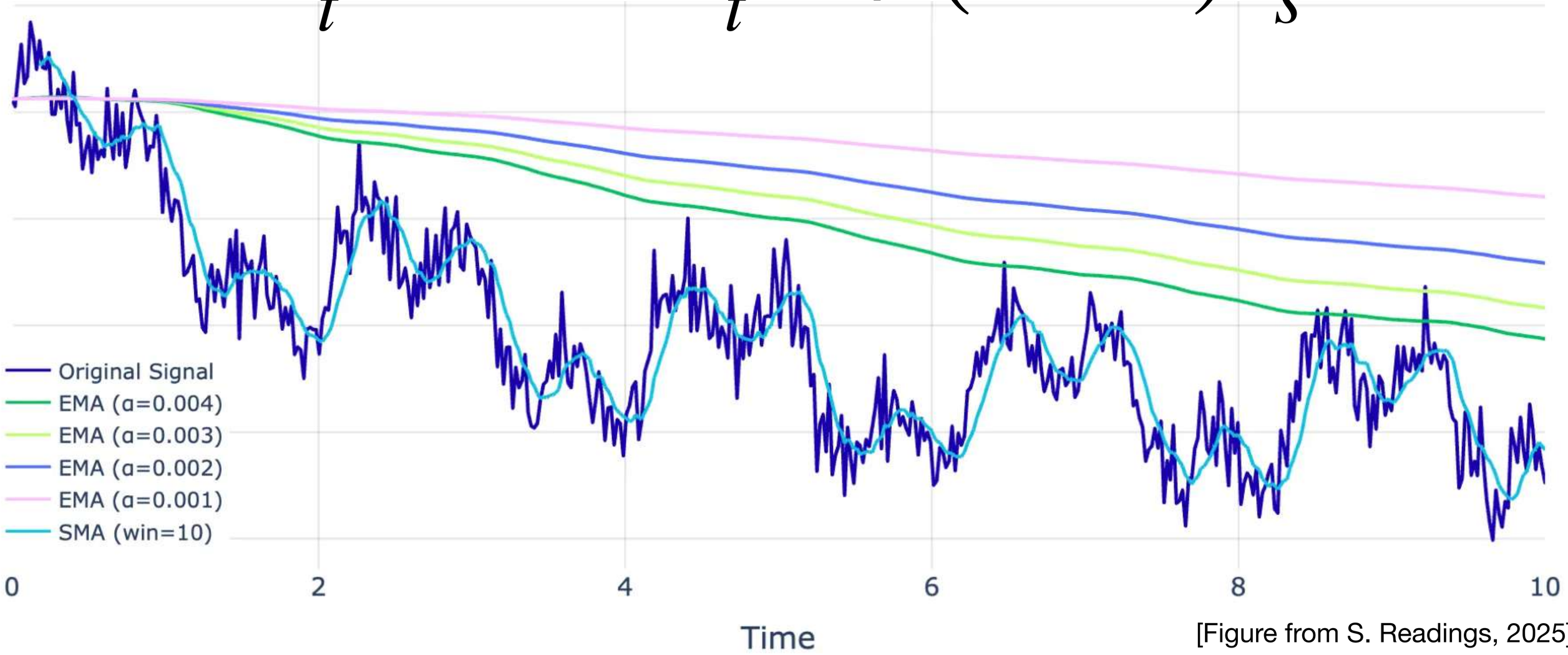
Exponential Moving Average (EMA)

$$\theta_t^{(k+1)} = \lambda \theta_t^{(k)} + (1 - \lambda)\theta_s^{(k)}$$

[Figure from S. Readings, 2025]

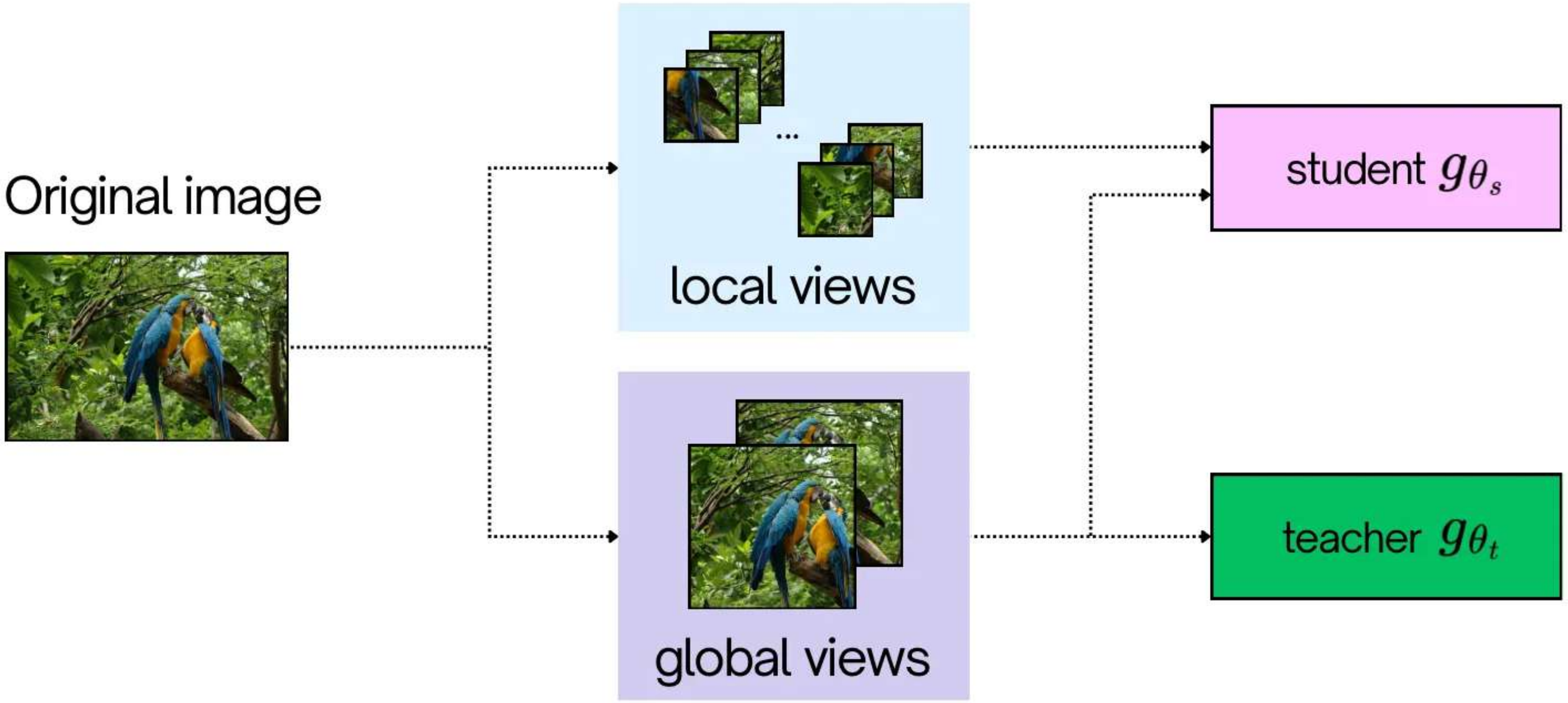
Exponential Moving Average (EMA)

$$\theta_t^{(k+1)} = \lambda \theta_t^{(k)} + (1 - \lambda)\theta_s^{(k)}$$



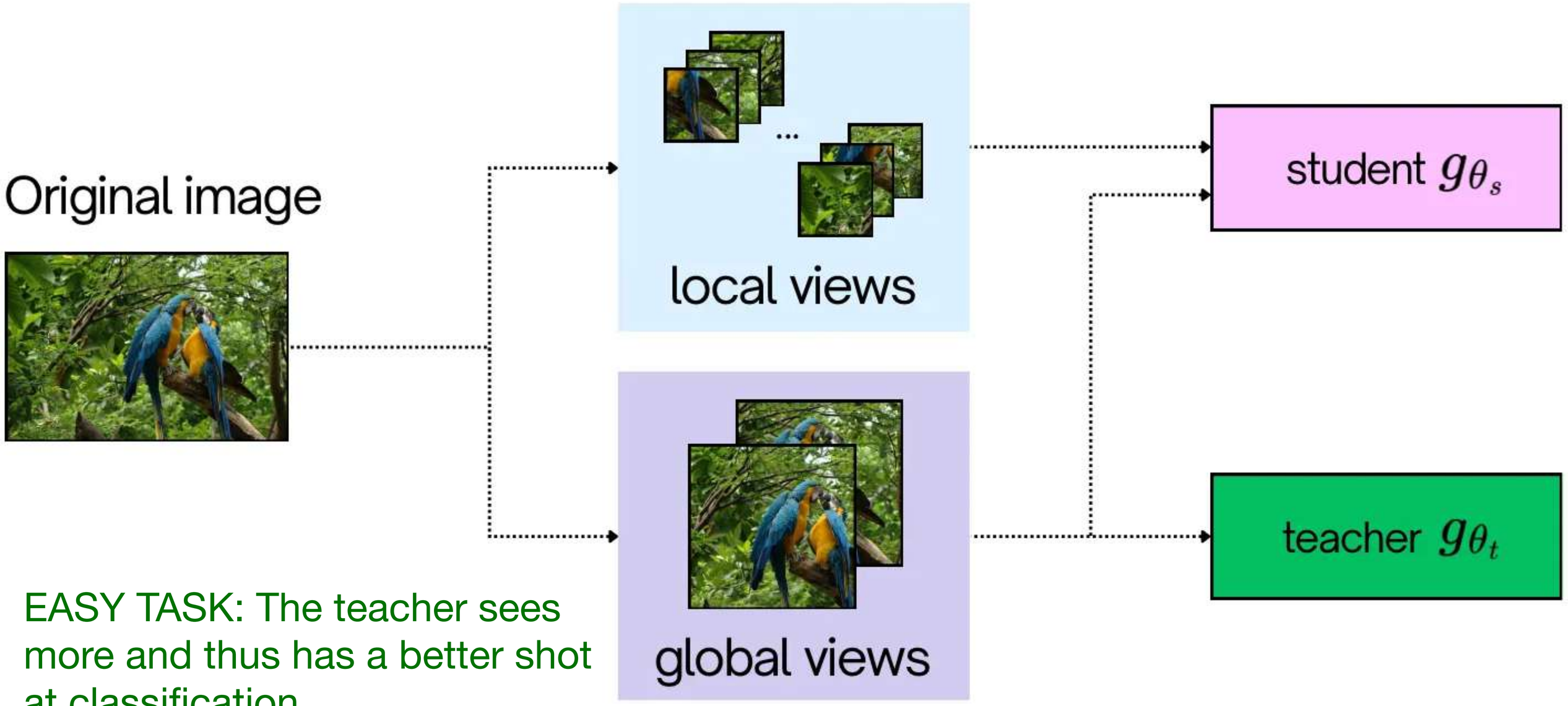
[Figure from S. Readings, 2025]

Easy vs difficult task - same label



[Figure from S. Readings, 2025]

Easy vs difficult task - same label

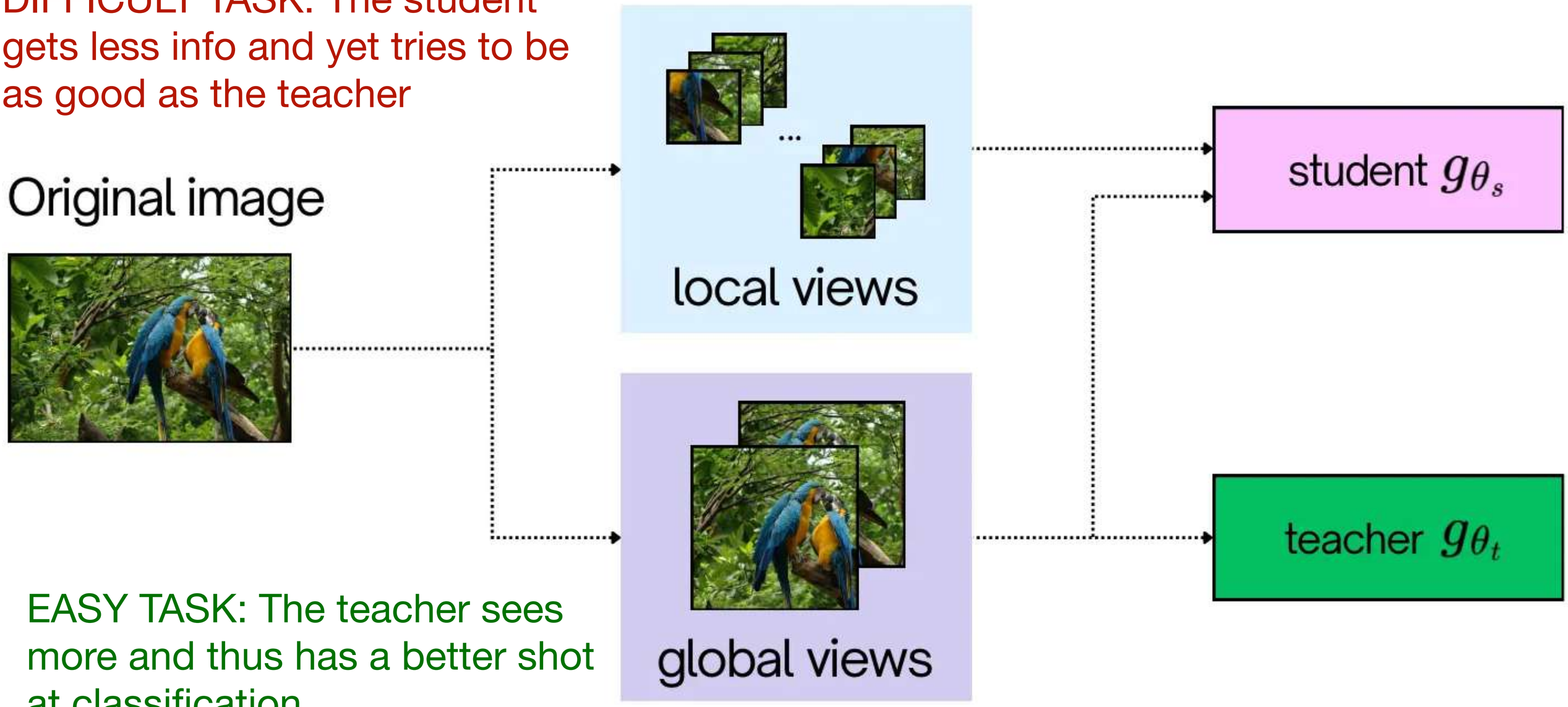


EASY TASK: The teacher sees more and thus has a better shot at classification

[Figure from S. Readings, 2025]

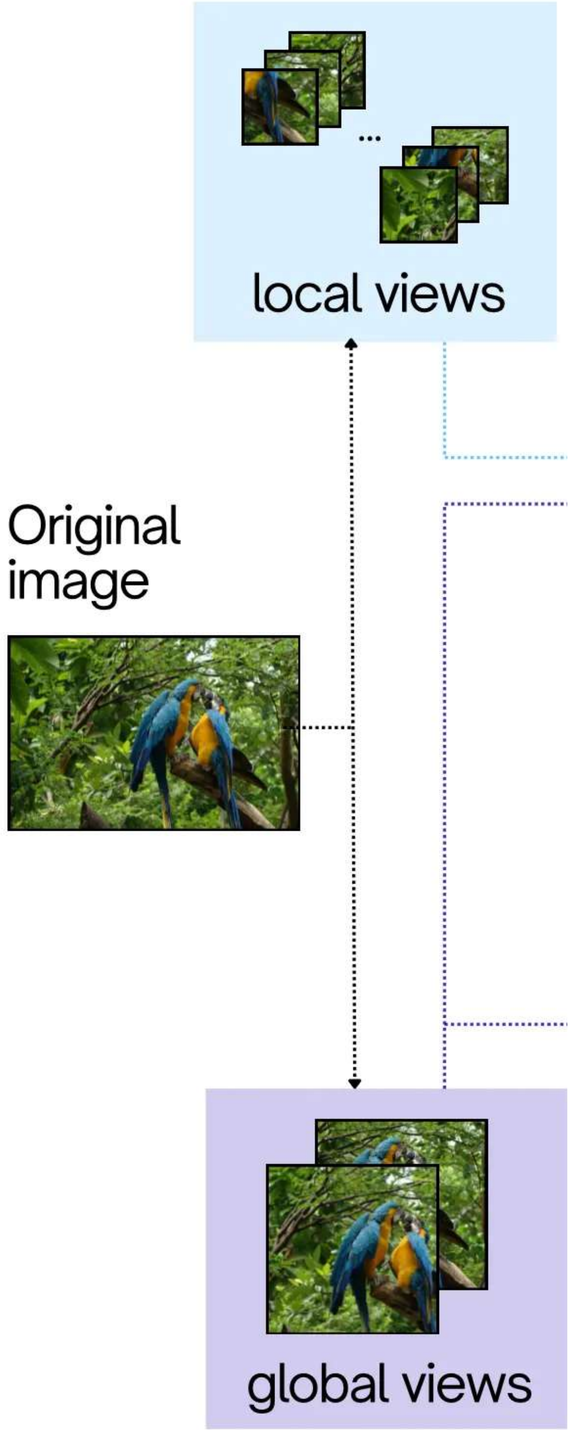
Easy vs difficult task - same label

DIFFICULT TASK: The student gets less info and yet tries to be as good as the teacher

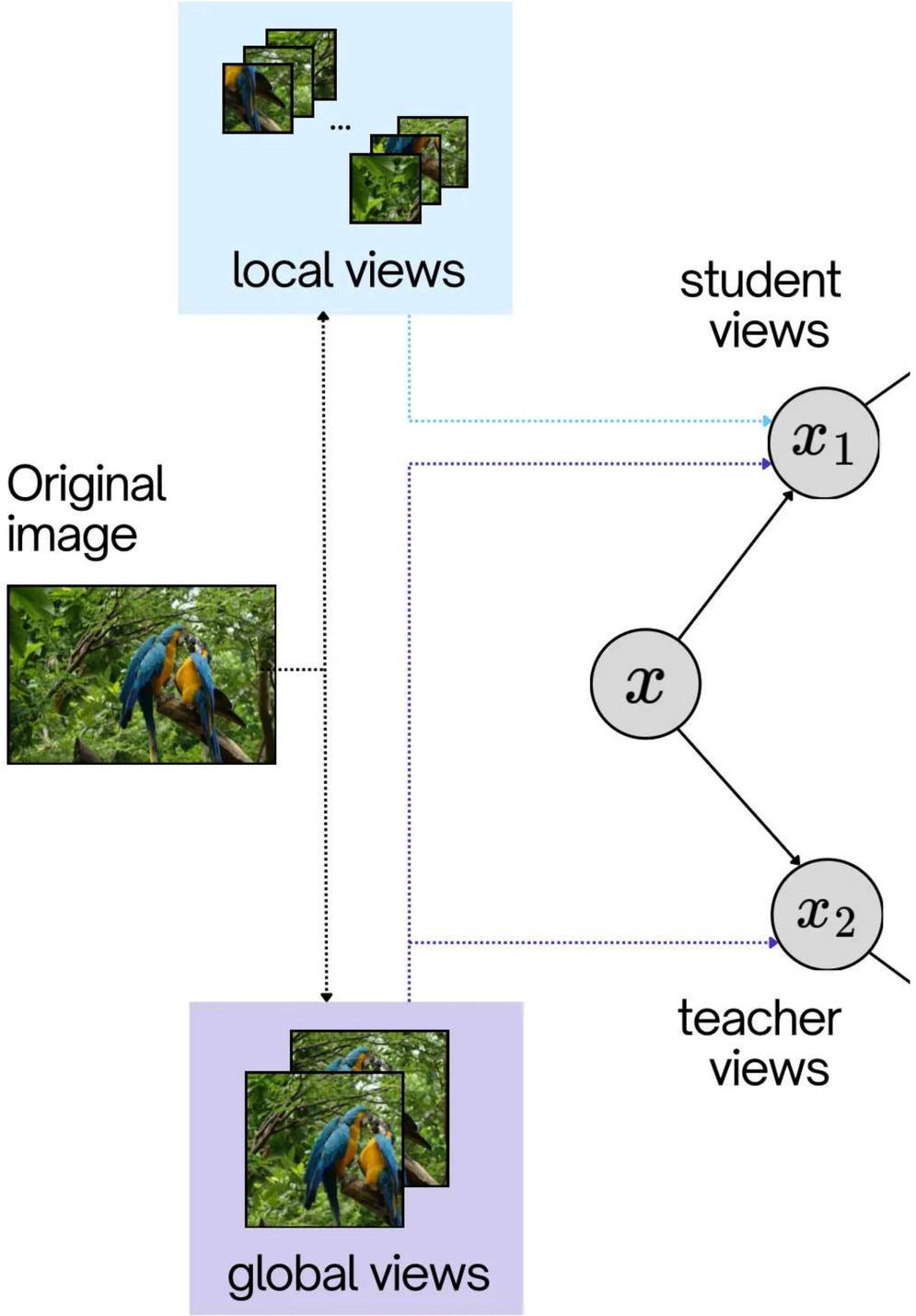


EASY TASK: The teacher sees more and thus has a better shot at classification

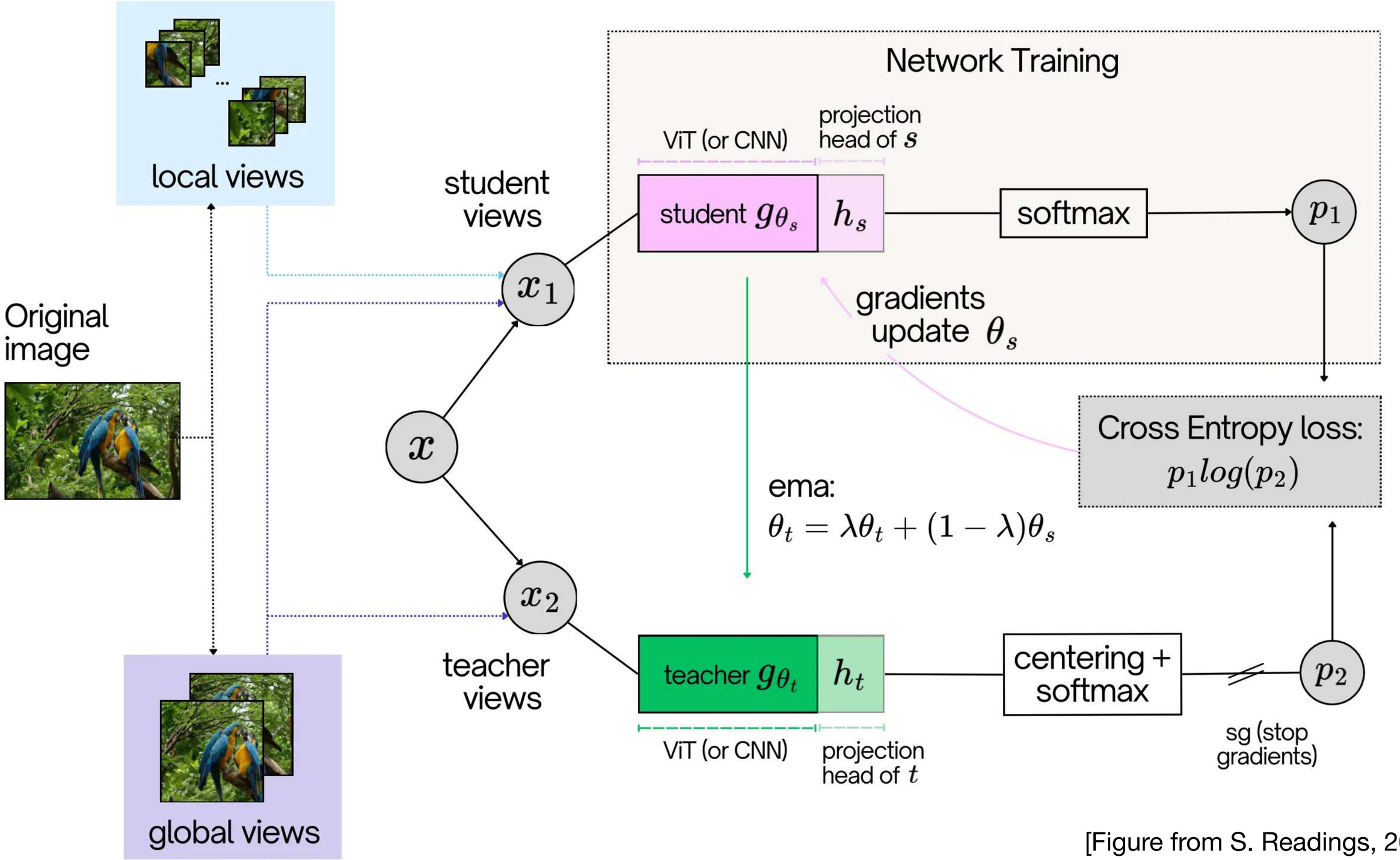
[Figure from S. Readings, 2025]



[Figure from S. Readings, 2025]



[Figure from S. Readings, 2025]



[Figure from S. Readings, 2025]

Centering

$$g_t(x) \leftarrow g_t(x) + c$$

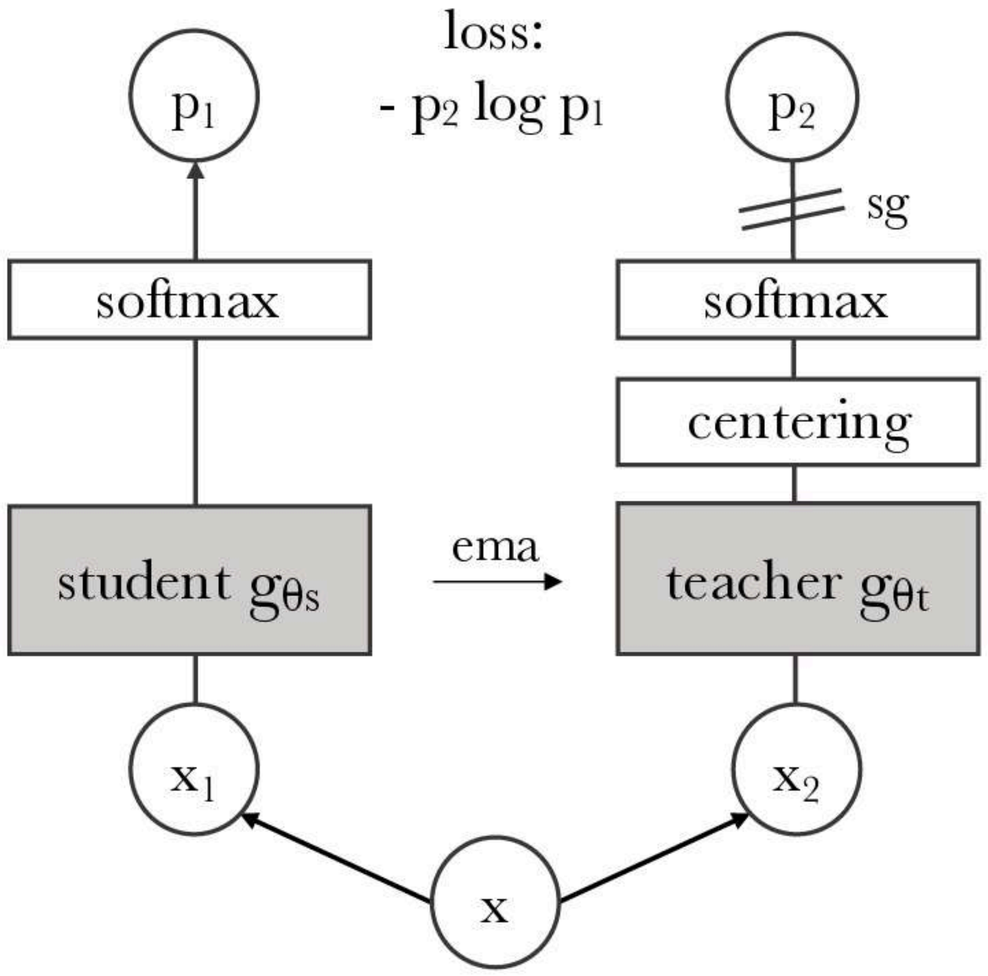
$$c \leftarrow mc + (1 - m)\left(\frac{1}{B}\sum_{i=1}^{B} g_{\theta_t}(x_i)\right)$$

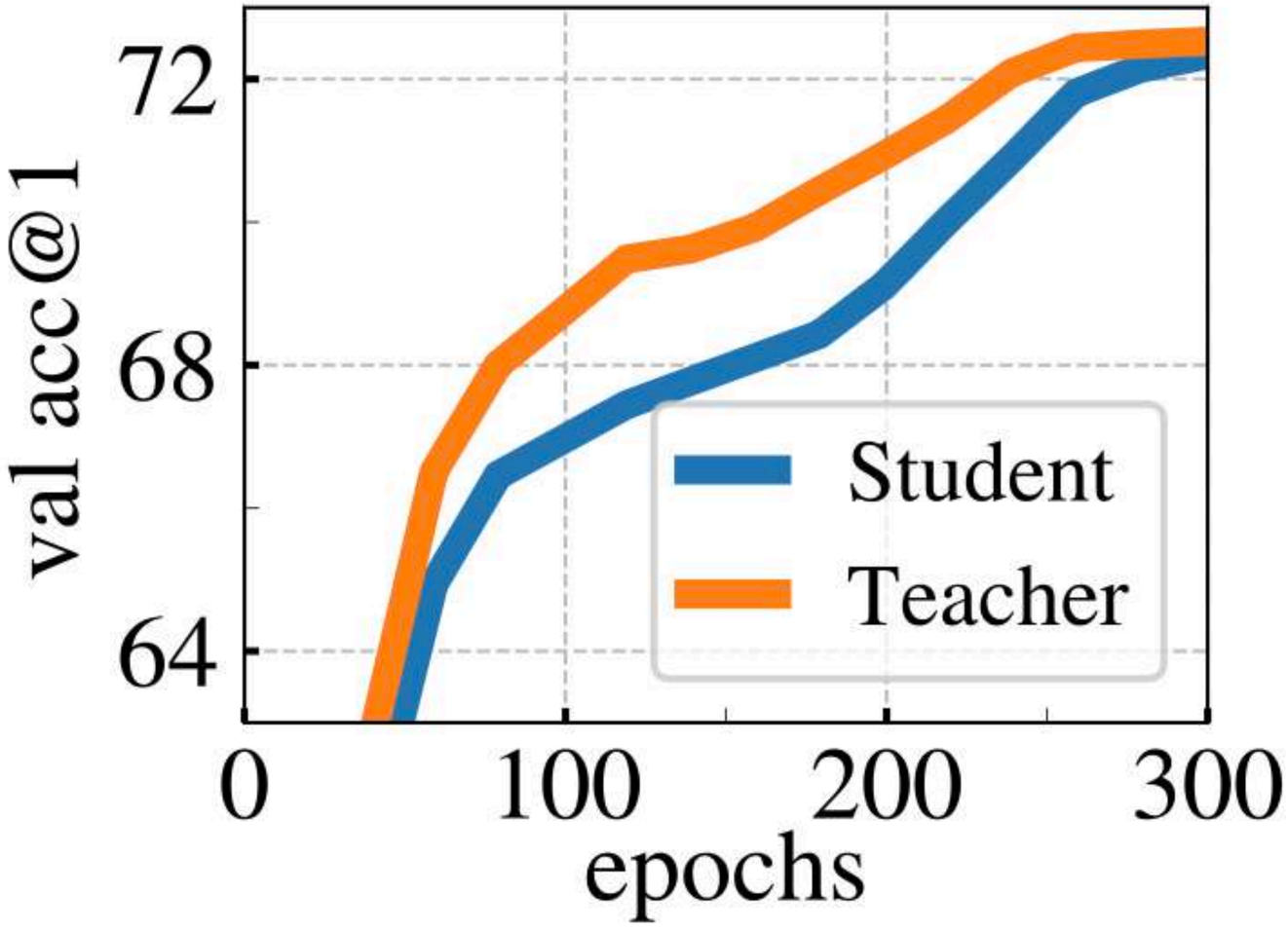
m = rate parameter

B = batch size

Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```





Teacher	Top-1
Student copy	0.1
Previous iter	0.1
Previous epoch	66.6
Momentum	72.8

Supervised

DINO